

Securing Edge System Accelerators: A Design Evaluation Using ARM Fast Models

CPSC 449 Undergraduate Honours Thesis

by

Kasra Kamal

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF SCIENCE (HONOURS)

in

The Faculty of Science

(Honours in Computer Science)

THE UNIVERSITY OF BRITISH COLUMBIA

(Vancouver)

April 2022

© Kasra Kamal, 2022

To Mom, Dad, & Sep

ABSTRACT

Edge computing is a new computing paradigm that performs computations at the edge of the network and is now being used for GPU-accelerated applications due to real-time application constraints. Securing GPU-accelerated applications running on edge devices is necessary to obtain data confidentiality and integrity. However, edge devices are vulnerable to various threats (e.g, malware) due to being distributed and having a large attack surface. The key primitive for protecting applications running on an untrusted platform stack are Trusted Execution Environments (TEEs). Existing TEEs however have focused on securing CPUs without securing GPUs.

We design a TEE for ARMv8-based devices with TrustZone security extensions and integrated GPUs. The TEE provides an isolation environment built in Normal World (NW) and provides secure request-level GPU access. We utilize the TrustZone provided TrustZone Address Space Controller (TZASC) to securely partition memory regions.

We demonstrate the applicability of our design by evaluating the overheads present using ARM Fast Models.

ACKNOWLEDGMENTS

I would like to acknowledge and give my warmest thanks to my supervisor Dr. Aastha Mehta who made this work possible. This thesis was a journey with many ups and downs. Her support and guidance carried me through it.

I would also like to thank Dr. Arpan Gujarati and Puneet Mehrotra who supported this work and provided detailed advice throughout the process.

Finally, I would like to thank my parents for their constant support and my brother for keeping me motivated and letting me bounce ideas off him.

CONTENTS

1	Introduction	1
1.1	Context	1
1.2	Problem Statement	2
1.3	Solution Overview	3
1.4	Contributions & Outline	4
2	Background	6
2.1	ARM Security Features	6
2.1.1	ARM Exception Levels	7
2.1.2	ARM TrustZone	7
2.1.3	TZASC	9
2.2	TEE Software Building Blocks	11
2.3	Securing Applications Using TrustZone-based TEE . .	11
2.4	GPU	14
2.4.1	Discrete Vs. Integrated GPUs	15
2.4.2	ARM Mali-G76 Overview	16
2.4.3	GPU Computing (OpenCL)	17
2.5	Current Hardware Options	20
2.5.1	NVIDIA Jetson TX2	20
2.5.2	HiKey 960	21
2.5.3	ARM Simulation Motivation	21
2.6	ARM Fast Models	22
2.6.1	FVP_Base_RevC-2xAEMvA Overview	22
2.6.2	GPU Fast Models	23
2.6.3	FVP Timing & Performance Evaluation	25
3	Design & Architecture	27
3.1	Design Goals	27
3.2	Threat Model	27
3.3	Design Overview	28
4	Implementation	35
4.1	Hardware Simulation Implementation	35
4.1.1	NSAID Core Granularity Extensions	36
4.1.2	GPU Fast Model Extensions	37
4.1.3	InstTrace	40
4.2	Software Implementation	40
4.2.1	TZASC Dynamic Configuration & Fault Handling	41
4.2.2	GPU Memory Access Signaling	43
4.2.3	Secure OP-TEE Accesses	45
4.3	Applications for TEE Evaluation	47

5	Evaluation	53
5.1	Microbenchmarks	53
5.1.1	Differences in NW & SW Instruction Counts	54
5.1.2	SW Instruction Overheads	55
5.2	Macrobenchmarks	57
5.3	Security analysis	60
5.3.1	Theoretical	61
5.3.2	Empirical NW TZASC Configuration	63
5.3.3	Empirical GPU TZASC Configuration	66
5.3.4	Empirical SW TZASC Configuration	68
6	Related Work	72
6.1	ARM CPU TEEs	72
6.2	ARM GPU TEEs	74
6.3	Other Platforms	74
6.4	Virtualization & Software-based Isolation	75
7	Conclusion & Future Work	77
7.1	Contributions Summary	77
7.2	Limitations	78
7.3	Future Work	79
	Bibliography	80

LIST OF FIGURES

Figure 2.1	ARM exception levels.	7
Figure 2.2	TrustZone overview.	8
Figure 2.3	TZC-400 overview with two filter units.	9
Figure 2.4	TZASC region mapping example.	10
Figure 2.5	OP-TEE shared memory buffer lifetimes.	14
Figure 2.6	OP-TEE NA to TA data flow.	14
Figure 2.7	Discrete GPU high-level block diagram.	15
Figure 2.8	Integrated GPU high-level block diagram.	16
Figure 2.9	ARM Mali-G76 GPU high-level block diagram.	17
Figure 2.10	Memory layout when using CL_MEM_ALLOC_HOST_PTR with clCreateBuffer.	19
Figure 2.11	Base Platform RevC block diagram.	23
Figure 2.12	libnomali.so architectural location within FVP_Base_RevC-2xAEMvA.	24
Figure 3.1	Design overview. Two NI-W instances are shown-cased. The diagram shows contiguous NI-W DRAM regions. This may not be the case and is a simplification of the diagram. Allowed memory accesses are shown with an arrow.	29
Figure 3.2	Memory region partitioning of NI-W memory regions during NI-W setup. Allowed memory accesses are shown with an arrow. The memory regions do not need to be contiguous and as a result we provide spacing between memory blocks.	32
Figure 4.1	CPUlabeller fast model functionality. CPUlabeller intercepts CPU core memory transactions to assign an appropriate NSAID.	37
Figure 4.2	LabellerForGPUProtMode fast model functionality. LabellerForGPUProtMode intercepts GPU memory transactions to assign an appropriate NSAID.	39
Figure 5.1	OP-TEE client API log base-10 transformed instruction counts.	57
Figure 5.2	Application latency of nw_cpu, nw_gpu, sw_cpu, and sw_gpu. Matrix sizes ranging from 16x16 to 512x512 are included. The matmul_execute step is not included.	59
Figure 5.3	Application latency of nw_cpu, nw_gpu, sw_cpu, and sw_gpu. Matrix sizes ranging from 16x16 to 256x256 are included.	60

LIST OF TABLES

Table 2.1	TZASC region control registers. <n> is the region number in the range 1 – 8.	10
Table 4.1	nw_cpu application steps.	50
Table 4.2	nw_gpu application steps.	51
Table 4.3	sw_cpu application steps.	51
Table 4.4	sw_gpu application steps.	52
Table 5.1	sw_cpu instruction count percent increase over nw_cpu per instruction group.	55
Table 5.2	OP-TEE client API function SMC calls.	57
Table 5.3	Latency components for nw_cpu, nw_gpu, sw_cpu, and sw_gpu.	58

LISTINGS

Listing 2.1	GlobalPlatform client example flow.	12
Listing 2.2	OpenCL example application, highlighting the OpenCL functions.	19
Listing 2.3	OpenCL API clCreateBuffer interface.	19
Listing 2.4	libnomali.so callback information structure.	24
Listing 2.5	FVP simulated time example.	25
Listing 2.6	FVP CPI example file.	26
Listing 4.1	Default FVP_Base_RevC-2xAEMvA NSAID configuration parameters.	36
Listing 4.2	CPUlabeller assigning core NSAIDs.	37
Listing 4.3	Extended FVP_Base_RevC-2xAEMvA NSAID configuration parameters, where we configure CPU NSAIDs at a core granularity.	37
Listing 4.4	libnomali.so register writing extensions.	39
Listing 4.5	LabellerForGPUProtMode assigning GPU NSAIDs.	39
Listing 4.6	InstTrace output file snapshot.	40
Listing 4.7	tzc Linux device driver TZASC configuration handler.	42
Listing 4.8	Mali GPU Linux device driver GPU memory access handler.	44
Listing 4.9	OP-TEE core GPU memory access handler.	45
Listing 4.10	Sequence of functions called by OP-TEE core during TA initialization, highlighting the TEE_MATTR_SECURE addition.	46

Listing 4.11	TEEC_AllocateSharedMemory API changes.	46
Listing 4.12	Matrix multiplication loop.	47
Listing 4.13	TA matrix multiplication function.	48
Listing 4.14	nw_gpu and sw_gpu OpenCL initialization. Error checking is removed.	49
Listing 4.15	GPU memory access signaling function nw_send_to_gpu.	50
Listing 4.16	GPU memory access signaling function sw_send_to_gpu.	50
Listing 5.1	TA_InvokeCommandEntryPoint immediately returns.	56
Listing 5.2	Application for multithreaded general TZASC configuration, highlighting the main function.	64
Listing 5.3	Application for multithreaded general TZASC configuration, highlighting the thread handlers.	65
Listing 5.4	Output of running the multithreaded application with no TZASC configuration.	65
Listing 5.5	Output of running the multithreaded application with TZASC configuration.	66
Listing 5.6	TZASC GPU tzc_region configuration.	67
Listing 5.7	GPU write access allowed FVP logs.	67
Listing 5.8	GPU write access prevented FVP logs.	67
Listing 5.9	GPU cache behavior during TZASC changes. . .	68
Listing 5.10	GPU cache behavior during NSAID changes. . .	68
Listing 5.11	TA attempting to access a shared memory region.	69
Listing 5.12	NA optionally preventing SW access to a region.	70
Listing 5.13	Output of allowing SW access to the shared memory region.	71
Listing 5.14	Output of preventing SW access to the shared memory region.	71

ACRONYMS

API	Application Programming Interface
ATF	ARM Trusted Firmware
AXI	Advanced eXtensible Interface
CPI	Cycles Per Instruction
CPU	Central Processing Unit
DECERR	Decode Bus Error
DRAM	Dynamic Random Access Memory
EL0	Exception Level 0
EL1	Exception Level 1

EL2	Exception Level 2
EL3	Exception Level 3
FPGA	Field Programmable Gate Array
FVP	Fixed Virtual Platform
GPU	Graphics Processing Unit
KM	Kernel Module
L1	First-Level Cache
L2	Second-Level Cache
ML	Machine Learning
MMU	Memory Management Unit
NA	NW Application
NI-A	Normal-Isolated Application
NI-L	Normal-Isolated Library
NI-W	Normal-Isolated World
NOS	NW OS
NS	Non-Secure
NSAID	Non-Secure Access IDentity
NW	Normal World
OP-TEE	Open Portable Trusted Execution Environment
OS	Operating System
OpenCL	Open Compute Library
PA	Physical Address
PCIe	Peripheral Component Interconnect Express
RA	Remote Attestation Application
SA	Sanctuary Application
SCR	Secure Configuration Register
SGX	Software Guard Extensions
SMC	Secure Monitor Call
SMMU	System Memory Management Unit
SW	Secure World
SoC	System on Chip
TA	Trusted Application
TCB	Trusted Compute Base
TEE	Trusted Execution Environment
TEEC	TEE Client
TOS	SW OS

TZASC	TrustZone Address Space Controller
TZMA	TrustZone Memory Adapter
TZPC	TrustZone Protection Controller
VA	Virtual Address
s-EL0	secure-EL0
s-EL1	secure-EL1

INTRODUCTION

1.1 CONTEXT

Traditional cloud computing is no longer sufficient to support the diverse computing needs of end-users. The International Data Corporation (IDC) predicts that the world's data is expected to grow to 175 zettabytes by 2025, with an annual growth rate of 61% [38]. Storing and accessing data in cloud data centers pushes network bandwidth requirements to their practical limits. Cloud data centers are unable to meet the transfer rates and response times required by many real-time applications. Additionally, uploading data to cloud data centers risks leaking private user data. To resolve both the latency and privacy issues present in cloud computing, edge computing has emerged as a replacement for traditional cloud computing tasks [16].

Edge computing is a new computing paradigm that performs computing at the edge of the network to bring computing closer to the data sources. Edge computing has wide applications ranging from real-time traffic updates in autonomous vehicles to supporting interactive user services, such as video conferencing and medical monitoring [16]. Many of these applications employ sophisticated Machine Learning (ML) techniques, which make them compute and resource intensive. Standard Central Processing Units (CPUs) present in traditional edge devices are unable to meet the computation requirements to support ML demands. Consequently, the next generation of edge devices are incorporating accelerators, such as Graphics Processing Units (GPUs) and Field Programmable Gate Arrays (FPGAs), which provide the nec-

essary parallel compute architectures [26]. We focus on GPU-based accelerators in this thesis.

1.2 PROBLEM STATEMENT

Securing applications running on edge devices is an important problem that, unlike public cloud data centers, has not received much attention [32]. Edge devices are however more vulnerable to various threats (e.g., malware) than cloud devices due to being distributed and having a large attack surface [36]. This makes the applications hosted on the edge devices vulnerable too.

The key primitive for protecting applications from a compromised platform stack (i.e., the privileged Operating System (OS)) is a Trusted Execution Environment (TEE), which is a secure region of a processor for executing an application code on data [35]. A Trusted Execution Environment (TEE) is an isolated execution environment that remains secure even if the system software on the device is compromised. TEEs are typically backed by a hardware root of trust provided by chip vendors, and provide confidentiality and integrity guarantees for the code and data inside the TEE. A large number of edge devices typically have ARM System on Chips (SoCs) and come with ARM's hardware security extensions, namely ARM TrustZone. TrustZone enables an application to be executed in one of two distinct "worlds" on a core — Normal World (NW) and Secure World (SW). In particular, when an application runs in SW, its execution cannot be affected by any code deployed in the NW.

Despite the strong guarantees, TrustZone has two main limitations. (1) Applications running inside SW need to rely on an OS in the SW for process isolation. However, a compromise in any part of the software stack can violate the security of the application. The size of the Trusted

Compute Base (TCB) for an application is still large. (2) SW applications cannot directly use additional compute elements on the platform, e.g., integrated GPUs. Using GPUs will require adding drivers to the SW OS, which further increases the size of the application's TCB.

1.3 SOLUTION OVERVIEW

To improve the security of GPU-accelerated edge applications, we design a TEE for edge devices spanning both the CPU and GPU with minimal overhead. We therefore design a TEE that provides only the essential additions to the TCB and requires minimal adjustments to existing GPU-accelerated applications. We describe a TEE for ARMv8-based devices with TrustZone security extensions and integrated GPUs. The TEE provides an isolation environment built in NW. The TEE additionally provides secure request-level GPU access. We utilize the TrustZone provided hardware primitive called TrustZone Address Space Controller (TZASC), to securely partition memory regions.

We implement the necessary software changes and minimal hardware changes required to support a full implementation of the TEE. For software changes, we implement the following features: (1) support for dynamic configuration of the TZASC from NW and SW, (2) signaling of GPU memory accesses from NW and SW within a simulation environment, and (3) modification of SW software to use secure memory accesses. For hardware changes, we implement the following features: (1) support for request-level GPU isolation by utilizing the TZASC, (2) simulation of GPU memory accesses within a simulation environment, and (3) addition of an instruction tracer that logs the sequence of CPU instructions executed.

We build a prototype of the TEE using ARM Fast Models [8], an open source simulator that enables rapid prototyping and evaluation of

custom ARM hardware designs. We present an empirical performance evaluation of our TEE prototype in ARM Fast Models. First, through systematic microbenchmarks, we analyze the latency costs of various components involved in the initialization and enforcement of the TEE isolation. Secondly, we present an end-to-end performance evaluation of an Open Compute Library (OpenCL)-based ML application that executes inside the TEE and leverages the secured GPU for acceleration. We additionally evaluate the security of the TEE by providing both a theoretical analysis and an empirical analysis. The empirical analysis consists of various scenarios to ensure aspects of the TEE design provide the necessary security guarantees. Overall, the evaluation demonstrates that the proposed TEE design is a practical solution.

1.4 CONTRIBUTIONS & OUTLINE

We make the following contributions.

- We provide a comprehensive background on the following in [Chapter 2](#).
 - ARM TrustZone, with an emphasis on the TZASC.
 - Current TEE architectures, including ATF and OP-TEE workflows.
 - GPU compute architectures, discussing discrete vs. integrated GPUs, an overview of ARM Mali-G76 GPU, and OpenCL GPU Computing.
 - Current embedded hardware options that provide TrustZone (with TZASC) and integrated GPUs.
 - ARM fast models and FVPs, GPU fast model limitations and extension mechanisms, and methods to evaluate FVPs.
- We describe a TEE design for GPU-accelerated applications on ARM edge devices with integrated GPUs. We build an isolation environment within NW and provide secure request-level GPU

access within the TEE. The design and architecture are detailed in [Chapter 3](#).

- We provide implementation details of the software and hardware extensions to support a full implementation of the TEE design. We additionally provide implementation details for the software and hardware components added to support systematic evaluation of overheads present in the design. The implementation details are described in [Chapter 4](#).
- We provide an evaluation of overheads that will be encountered in the full TEE implementation described in [Chapter 3](#). This includes overheads present in world switches, memory region partitioning using TZASC, and secure GPU handling. We construct several scenarios to compare and evaluate the present overheads, which will directly resemble overheads present in particular steps of the full TEE implementation. We additionally evaluate the security of the TEE by providing both a theoretical analysis and an empirical analysis. We discuss evaluation results in [Chapter 5](#).
- We discuss related work in the field of TEE architectures in [Chapter 6](#).
- We conclude with a discussion on the limitations in the current work and future directions in [Chapter 7](#).

BACKGROUND

In this chapter, we first provide a background on the relevant details of the ARMv8 reference architecture and GPU architecture that are relevant for understanding the TEE design ([Section 2.1](#), [Section 2.2](#), [Section 2.3](#), and [Section 2.4](#)). We then survey the hardware specifications of several commercially available platforms and discuss the challenges of building a TEE on real hardware ([Section 2.5](#)). Finally, in [Section 2.6](#), we give an overview of ARM Fast Models, an open source simulator from ARM, which we use for demonstrating the viability of our TEE design.

We begin by discussing ARM TrustZone in [Section 2.1](#). We describe the TZASC in detail in [Section 2.1.3](#). We discuss the software components built on top of the hardware primitives to provide a TEE in [Section 2.2](#) and [Section 2.3](#). We discuss GPU fundamentals, paying particular attention to the memory model in [Section 2.4](#). We survey current hardware options that provide the mentioned security extensions in [Section 2.5](#) and discuss the need for simulations in [Section 2.6](#).

2.1 ARM SECURITY FEATURES

We discuss ARM exception levels in [Section 2.1.1](#). We discuss ARM TrustZone fundamentals in [Section 2.1.2](#) and we describe in detail the TZASC in [Section 2.1.3](#).

2.1.1.1 ARM Exception Levels

In the following sections and chapters, we use the ARMv8 architecture running in the AArch64 execution state. ARMv8 provides several different exception levels referred to as EL<n>, with n in the range 0-3. Each exception level has a different level of access to system and processor resources. For example, the lowest level of privilege is referred to as Exception Level 0 (EL0). [Figure 2.1](#) showcases the different exception levels. Since Exception Level 2 (EL2) is not used in our design, we do not discuss it in detail. For more details on the ARMv8 AArch64 exception model, we refer the interested reader to [\[11\]](#).

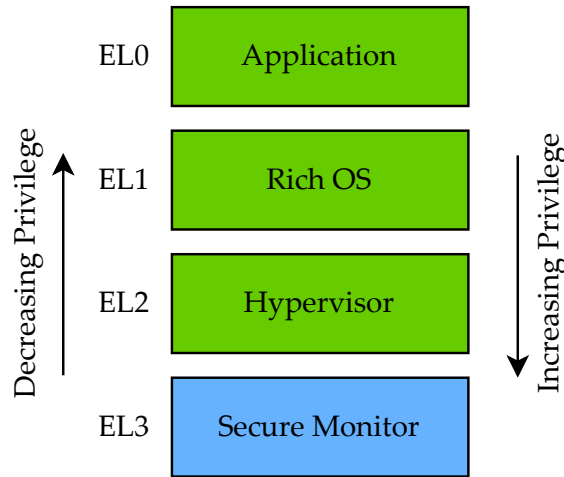


Figure 2.1: ARM exception levels.

2.1.1.2 ARM TrustZone

ARM TrustZone is a hardware security extension based on the idea of partitioning a SoC CPU hardware and software into two worlds: SW and NW. [Figure 2.2](#) illustrates these concepts. The SW is everything that runs when the processor state is secure, and NW is everything that runs when the processor is in the Non-Secure (NS) state. Hardware barriers are established to prevent NW components from accessing SW

resources; the SW is not restricted. Specifically, the memory system prevents the NW from accessing i) secure physical regions; ii) system controls that apply to the SW; and iii) state switching outside of a small number of approved mechanisms. The SW enables the construction of an isolated programmable environment that can run a wide range of security applications.

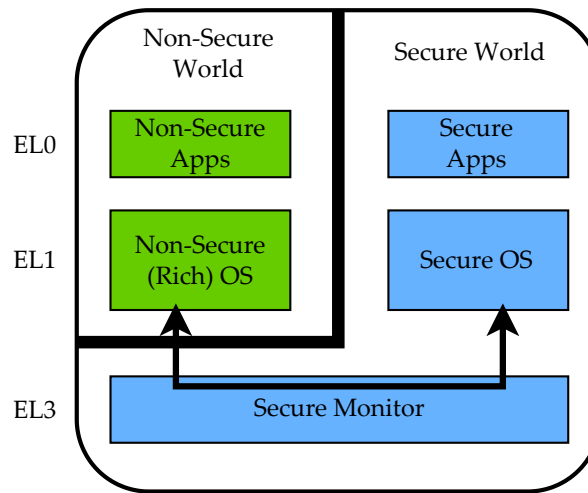


Figure 2.2: TrustZone overview.

The world where the processor currently executes in is determined by the NS bit, which can be read from the Secure Configuration Register (SCR), and is present with all memory transactions. TrustZone introduces an additional mode called the monitor mode, which is used to run a secure monitor that preserves the processor state and issues world transitions. The monitor mode acts independently of the value of the NS bit. A privileged instruction—Secure Monitor Call (SMC)—accessible from Exception Level 1 (EL1) and Exception Level 3 (EL3), allows for software running in either world to communicate with the secure monitor and to context switch to either world.

TrustZone extends the memory infrastructure with additional security peripherals, including TZASC, TrustZone Memory Adapter (TZMA), and TrustZone Protection Controller (TZPC). We discuss TZASC in [Section 2.1.3](#). For more details on TrustZone features, including an

analysis of TZMA and TZPC, we refer the interested reader to [35] and [37].

2.1.3 TZASC

The TZASC allows dynamic classification of Advanced eXtensible Interface (AXI) slave memory-mapped devices as secure or NS. We describe TZASC version TZC-400 [4]. Controlled by the SW, the TZASC allows partitioning of a single Dynamic Random Access Memory (DRAM) memory unit rather than requiring separate secure and NS units. The TZASC performs security checks on transactions to memory or peripherals. Up to eight separate regions in the physical address space can be created, each with separate security level settings. Memory transactions must meet the security requirements to gain access to the memory or peripheral. Region permissions are configured separately for read and write transactions. Figure 2.3 shows a high-level view of the TZASC intercepting memory transactions, where filter units perform the security checks.

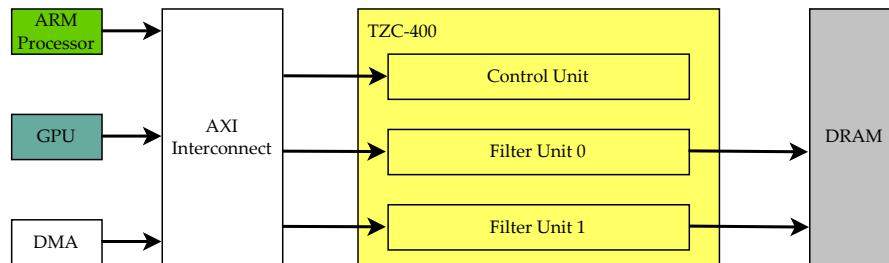


Figure 2.3: TZC-400 overview with two filter units.

TZASC region permissions are configured separately for NW accesses and SW accesses. For SW accesses, a region must be explicitly configured to allow secure access if required, regardless of the NW settings. This is done for a region by setting the read and/or write enable bits in the `REGION_ATTRIBUTES_<n>` register. For NW accesses, TZASC uses the Non-Secure Access IDentity (NSAID) input, which identifies the master that is the source of the transaction, to determine whether access

should be granted. TZC-400 supports up to 16 NSAIDs, and a region must be explicitly configured to allow NW access from a particular NSAID. This is done for a region by setting the read and/or write NSAID enable bits in the REGION_ID_ACCESS_<n> register.

Table 2.1 shows the region control registers which control the address space each region controls and the security attributes to use for that region.

REGISTER	DESCRIPTION
REGION_BASE_LOW_<n>	Region base address lower 32-bits
REGION_BASE_HIGH_<n>	Region base address upper 32-bits
REGION_TOP_LOW_<n>	Region top address lower 32-bits
REGION_TOP_HIGH_<n>	Region top address upper 32-bits
REGION_ATTRIBUTES_<n>	SW permissions
REGION_ID_ACCESS_<n>	NW permissions based on NSAID

Table 2.1: TZASC region control registers. <n> is the region number in the range 1 – 8.

Figure 2.4 shows an example region mapping configuration. Region 1 can only be accessed in NW with NSAID 1, region 2 can only be accessed in NW with NSAID either 1 or 2, and region 3 can only be accessed in SW. For example, if we assign masters ARM core0 to NSAID 1 and GPU to NSAID 2, ARM core0 in NW can access both region 1 and region 2 while GPU can only access region 2. Neither masters can access region 3 when in NW. Note that in real hardware, NSAIDs are shared by all cores in a cluster, where a cluster in this context is a group of ARM cores.

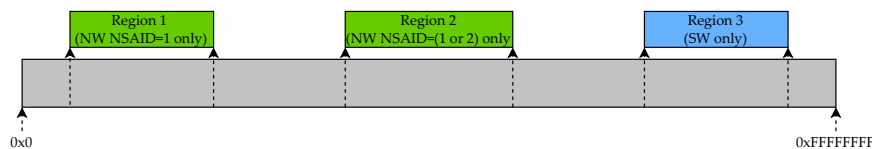


Figure 2.4: TZASC region mapping example.

2.2 TEE SOFTWARE BUILDING BLOCKS

ARM Trusted Firmware (ATF) provides a reference implementation of a secure monitor executing at EL3. Among other tasks, The ATF is responsible for handling all SMC calls and implements the SMC calling convention [13]. Therefore, the ATF contains handlers for context switching between NW and SW. Refer to the official documentation for more details [31].

Open Portable Trusted Execution Environment (OP-TEE) is an OS that runs at EL1 in the SW and enables executing applications inside the SW with few modifications. OP-TEE provides a reference implementation for a SW OS (TOS) and implements the GlobalPlatform standard set of Application Programming Interfaces (APIs) [20]. OP-TEE is divided into an OP-TEE core, running in secure-EL1 (s-EL1), and Trusted Applications (TAs), running in secure-EL0 (s-EL0). Refer to the official documentation for more details [30].

We give an overview of how to setup an application inside SW and how to invoke it using the GlobalPlatform APIs in [Section 2.3](#).

2.3 SECURING APPLICATIONS USING TRUSTZONE-BASED TEE

TruZtZone is aimed towards the creation of TEEs. Applications rely on an OS and several libraries, which often consist of thousands to millions of lines of code. Such a large code base is prone to bugs and vulnerabilities and typically exposes a large attack surface. TrustZone provides a hardware mechanism to build a TEE in which the software TCB can be several orders of magnitude smaller. With ARM TrustZone, an application can be secured by running in the SW on a core. The TrustZone hardware primitives—TZASC and the NS bit—together

with the secure monitor code enable a TEE inside the SW to run an application in isolation from the OS and libraries. Upon system boot the processor initially enters SW to perform security-related initialization, such as securing memory regions and peripherals, before context switching to NW to boot the NW OS (NOS). The NOS running in NW must issue SMCs to context switch and communicate with the SW. A compromised NOS is unable to access SW configured memory regions and therefore does not need to be trusted.

Throughout the following chapters, we will refer to several GlobalPlatform API functions. We therefore provide an overview of the relevant functions here, all of which correspond to the TEE Client (TEEC) API specification v1.0 [21]. OP-TEE implements the TEEC API specification in the library `libteec.so`. The functions and the order in which they are called is demonstrated in [Listing 2.1](#).

```
void main()
{
    // Create connection between NA and OP-TEE core
    TEEC_InitializeContext(...);

    // Create session between NA and TA
    TEEC_OpenSession(...);

    // Invoke TA with a specific command
    TEEC_InvokeCommand(...);

    // Close session between NA and TA
    TEEC_CloseSession(...);

    // Close connection between NA and OP-TEE core
    TEEC_FinalizeContext(...);
}
```

Listing 2.1: GlobalPlatform client example flow.

`TEEC_InitializeContext` initializes a new TEE context, forming a connection between a NW Application (NA) and OP-TEE. `TEEC_OpenSession` opens a new session between the NA and a specified TA. `TEEC_InvokeCommand` invokes the TA, optionally passing arguments to the TA. This is a complex function and is explained further in following

paragraph. `TEEC_CloseSession` closes the session between the NA and the TA. `TEEC_FinalizeContext` closes the connection between the NA and OP-TEE.

`TEEC_InvokeCommand` may optionally take an operation payload. The payload may either directly contain `uint32_t` values or may refer to a shared memory region `TEEC_SharedMemory`. The shared memory region can either be registered using `TEEC_RegisterSharedMemory` or allocated using `TEEC_AllocateSharedMemory`. Calling `TEEC_RegisterSharedMemory` requires an already user-allocated memory region, for example by calling `malloc`. `TEEC_AllocateSharedMemory` instead allocates a new block of memory from a special reserved physical NS DRAM region, denoted NS static shared memory. In OP-TEE, the NS static shared memory region is determined statically during compilation. The flow differences between `TEEC_RegisterSharedMemory` and `TEEC_AllocateSharedMemory` are highlighted in [Figure 2.5](#). OP-TEE attempts to perform zero-copy data transfers such that the shared memory region is directly mapped into the TA memory space. This is not possible when a TA does not have access to the same physical memory as the NA. `TEEC_AllocateSharedMemory` increases the likely-hood of zero-copy transfers since the NS static shared memory region is accessible by TAs (refer to [21] for details). [Figure 2.6](#) shows the overall data flow between NAs and TAs.

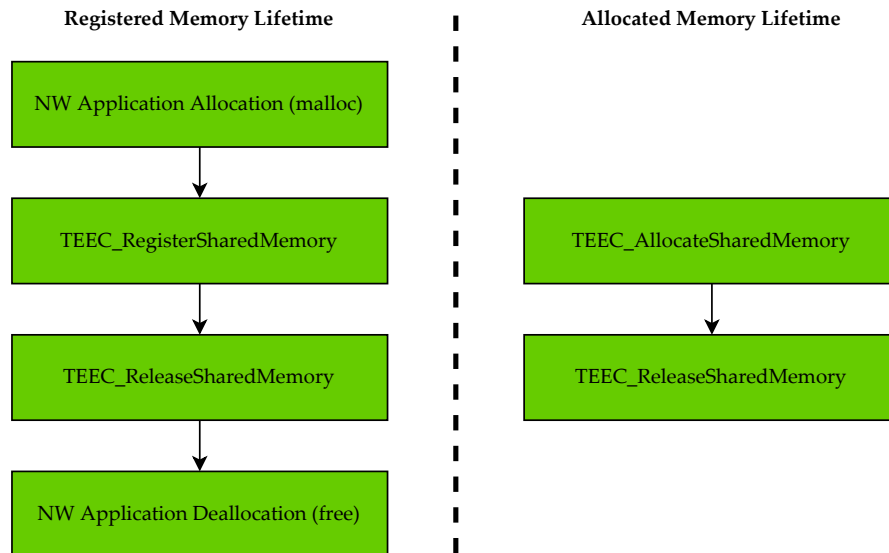


Figure 2.5: OP-TEE shared memory buffer lifetimes.

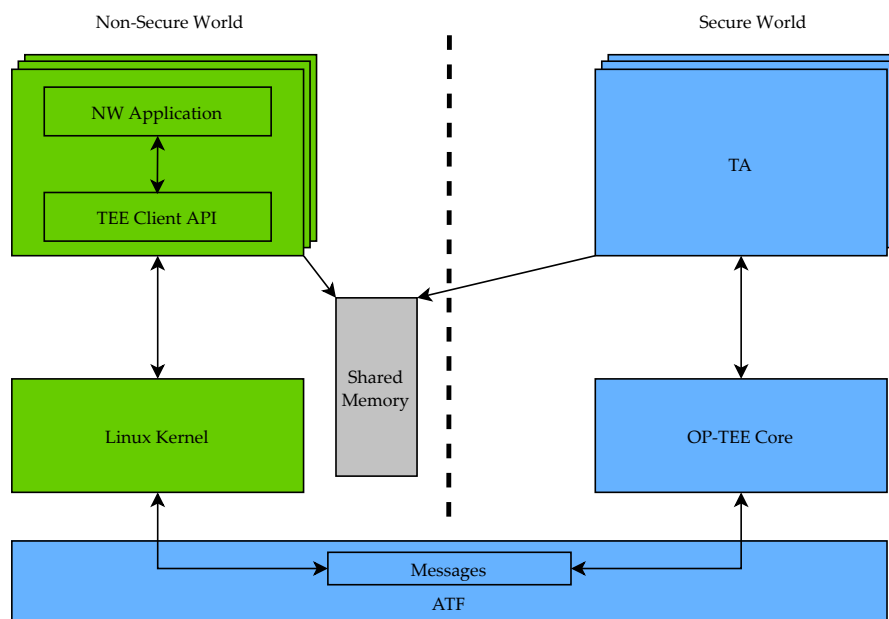


Figure 2.6: OP-TEE NA to TA data flow.

2.4 GPU

GPUs are modern computational devices that can be used to accelerate computer graphics, ML, graph computations, and various other workloads. A high throughput is achieved by including a large number of

computation cores working in parallel, resulting in high performance for certain workloads. There are two classes of GPU devices: discrete and integrated. We compare discrete GPUs and integrated GPUs in [Section 2.4.1](#). In the following chapters we use a simulated ARM Mali-G76 GPU. We therefore provide an overview of the ARM Mali-G76 GPU in [Section 2.4.2](#). We discuss GPU computing, focusing on OpenCL, in [Section 2.4.3](#).

2.4.1 Discrete Vs. Integrated GPUs

A typical discrete GPU generally consists of a large shared memory, a number of compute units, and a number of command dispatchers. Compute units typically contain private caches and a number of processing cores. [Figure 2.7](#) illustrates a high-level block diagram of a discrete GPU.

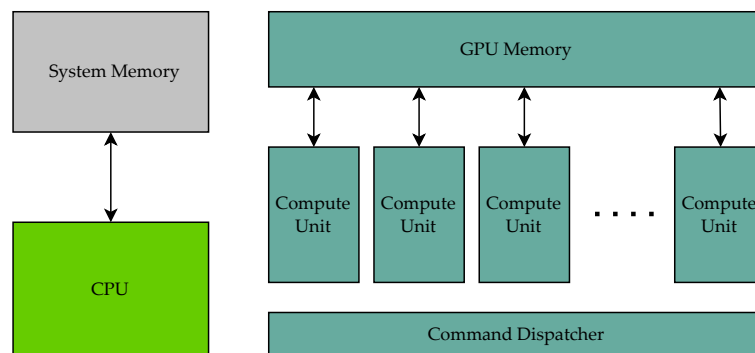


Figure 2.7: Discrete GPU high-level block diagram.

Discrete GPUs are generally added as expansion cards and most commonly communicate over the Peripheral Component Interconnect Express (PCIe) bus. As previously mentioned, discrete GPUs typically contain private GPU memory, separate from the system memory. However, expansion cards generally take up a fair amount of space, and require a separate power supply. This limits the applicability of discrete GPUs for use in resource constraint edge devices, which instead use integrated GPUs.

Integrated GPUs are built-in on the same SoC as the CPU and use system memory directly over the memory bus, rather than communicating over PCIe. Integrated GPUs must be small enough to fit on a single SoC and are therefore typically less powerful than their discrete GPU counterparts. Integrated GPUs also share system memory and power supply with the CPU. [Figure 2.8](#) illustrates a high-level block diagram of an integrated GPU.

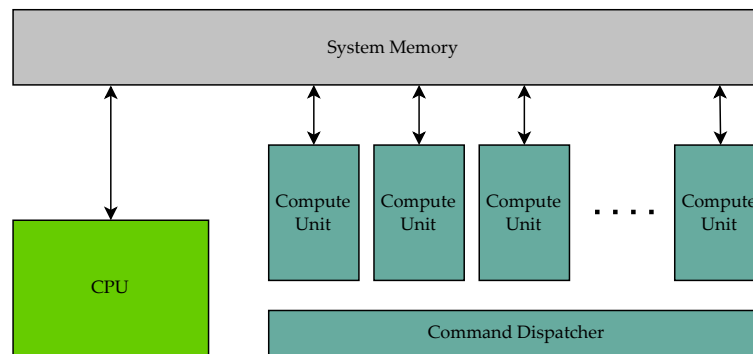


Figure 2.8: Integrated GPU high-level block diagram.

Examples of discrete GPU deployments include NVIDIA TITAN RTX [34] and AMD Radeon RX [2], both of which are built as expansion cards. Examples of integrated GPU deployments include NVIDIA Jetson TX2 ([Section 2.5.1](#)) and HiKey 960 ([Section 2.5.2](#)).

2.4.2 ARM Mali-G76 Overview

ARM refers to compute units in GPUs as shader cores. [Figure 2.9](#) illustrates a high-level block diagram of the ARM Mali-G76 GPU. A unified shader core architecture is used, which means that the design only includes a single type of hardware shader processor. The single shader processor can execute various types of workloads, including compute kernels. The Memory Management Unit (MMU) is programmed by the CPU by writing to MMU-related registers and translates Virtual Addresss (VAs) to Physical Addresss (PAs). Buffer memory regions (and other information) are sent to the GPU through the MMU translation table registers, `ASn_TRANSTAB_LO` and `ASn_TRANSTAB_HI`. The GPU MMU

serves a similar function to the ARM System Memory Management Unit (SMMU), and optionally both the GPU MMU and SMMU may be used for translating VAs to PAs. Refer to [5, 14] for more details on the ARM Mali-G76 architecture.

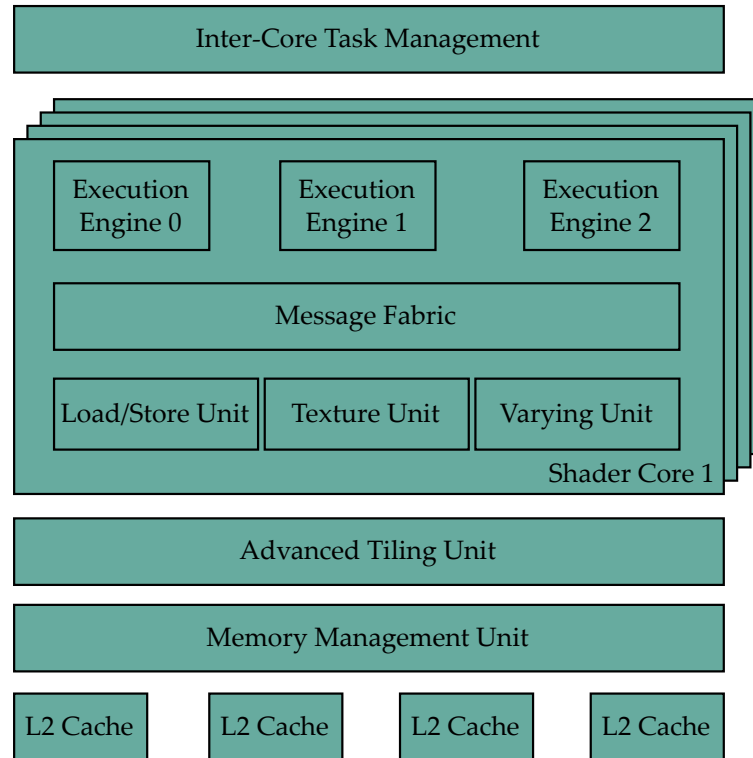


Figure 2.9: ARM Mali-G76 GPU high-level block diagram.

2.4.3 GPU Computing (OpenCL)

OpenCL is an open standard that provides parallel processing capabilities on a variety of accelerators, including GPUs. ARM provides a Mali GPU OpenCL driver for performing accelerated GPU computations [12]. OpenCL applications consist of two parts: host-side code and OpenCL kernels. The host-side code manages the execution of kernels by setting up command queues for memory transfer commands, kernel execution commands, and synchronization commands. The host application additionally defines a context for the kernels to run in, which consists of the kernels, compute devices (GPU), program objects, and memory objects. The kernels run on compute devices. A kernel is a

block of code that executes on compute units in parallel with other instances of the kernel. The host application submits the kernel for execution and additionally defines a multidimensional array of kernel instances to create. [Listing 2.2](#) shows an example OpenCL application, highlighting the OpenCL APIs that are called. Due to the importance of the OpenCL API `clCreateBuffer` in regards to allocated memory regions, we provide an overview of the function in the following paragraph. Refer to [\[23\]](#) for more details regarding the OpenCL API.

`clCreateBuffer` creates a buffer object corresponding to a OpenCL context. The function interface is specified in [Listing 2.3](#). An application may optionally set `host_ptr` to an already allocated memory region. Several allocation methods are possible and are specified through the `flags` bit-field. We describe the `CL_MEM_ALLOC_HOST_PTR` allocation flag since we use this flag in subsequent chapters. `CL_MEM_ALLOC_HOST_PTR` provides a hint to the device driver indicating that the buffer allocated will also be accessed from the host. This is the only method that avoids copying memory, thereby allowing both the host CPU and GPU to access the same memory region. This is shown in [Figure 2.10](#), where both the CPU and GPU access the same memory region allocated by `clCreateBuffer`.

```

int main()
{
    // Create OpenCL context
    clCreateContext(...);

    // Build kernel program
    clBuildProgram(...);

    // Create input and output buffers
    clCreateBuffer(...); clCreateBuffer(...);

    // Create a command queue for submitting commands
    clCreateCommandQueue(...);

    // Create kernel object
    clCreateKernel(...);

    // Add arguments to the kernel
    clSetKernelArg(...); clSetKernelArg(...);

    // Enqueue the kernel execution to the command queue
    clEnqueueNDRangeKernel(...);

    // Read the kernel's output
    clEnqueueReadBuffer(...);

    // Deallocate resources
}

```

Listing 2.2: OpenCL example application, highlighting the OpenCL functions.

```

cl_mem clCreateBuffer ( cl_context context,
                        cl_mem_flags flags,
                        size_t size,
                        void *host_ptr,
                        cl_int *errcode_ret )

```

Listing 2.3: OpenCL API `clCreateBuffer` interface.

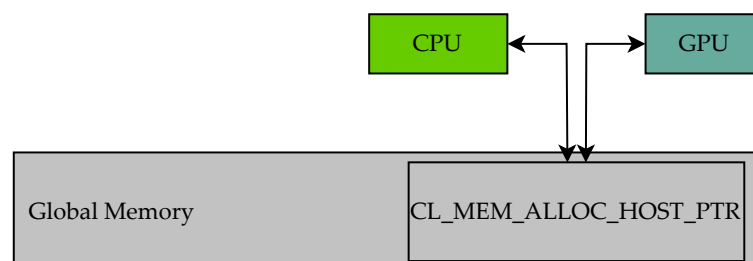


Figure 2.10: Memory layout when using `CL_MEM_ALLOC_HOST_PTR` with `clCreateBuffer`.

OpenCL provides an abstract memory model with several classes of memory. Private memory can only be accessed by a single kernel instance. Local memory can be accessed by a group of kernel instances and is specified using the `_local` keyword when declaring kernel variables. Global memory is accessible by all kernel instances executing within an OpenCL context and is specified using the `_global` keyword when declaring kernel variables. Many GPUs provide a separate shared memory cache within the GPUs for handling local memory. For example, NVIDIA Jetson TX2 provides a shared memory cache for a group of streaming processors [33]. Mali GPUs instead provide a unified memory system where private memory, local memory, and global memory are all allocated on system DRAM [5].

2.5 CURRENT HARDWARE OPTIONS

We provide an overview of several ARM SoC hardware platforms currently available. We discuss the ARM TrustZone features and integrated GPU peripherals that each hardware platform provides. We discuss NVIDIA Jetson TX2 and HiKey 960 in sections [Section 2.5.1](#) and [Section 2.5.2](#), respectively. We then motivate the need for simulating the ARM architecture in [Section 2.5.3](#), which will lead to the discussion of ARM Fast Models in [Section 2.6](#).

2.5.1 NVIDIA Jetson TX2

NVIDIA Jetson TX2 [33] is a fast, power-efficient embedded computing device. The module contains two CPU clusters, where one cluster consists of a dual-core NVIDIA Denver 2 processor and the other cluster consists of a quad-core ARM Cortex-A57 MPCore Processor. The module additionally contains a 256-core NVIDIA Pascal integrated GPU, capable of running high-performance CUDA (Compute Unified Device Architecture) computations. However, NVIDIA Jetson TX2

contains a custom build TrustZone architecture and does not contain a standard TZASC matching the ARM specification. Additionally, minimal documentation is provided for using their custom build TrustZone architecture. This limits the applicability of using this module in the following chapters.

2.5.2 *HiKey 960*

HiKey 960 [1] is a mobile development platform with the latest technologies from ARM and HiSilicon. The module uses a big.LITTLE ARM architecture consisting of two CPU clusters. One cluster consists of a quad-core ARM Cortex-A73 processor and the other cluster consists of a quad-core ARM Cortex-A53 processor. The module additionally contains an ARM Mali-G71 GPU, capable of performing OpenCL computations. HiKey 960 supports TrustZone extensions and includes a standard TZASC. However, NSAIDs can only be configured at the cluster granularity, which is too coarse-grained for enabling per-request isolation with high throughput. Consequently, prior systems [15, 32] that have relied on this feature for isolation are insufficient to provide fine-grained isolation at scale.

2.5.3 *ARM Simulation Motivation*

As we discussed, current available hardware platforms have limitations in providing both a TrustZone with a standard TZASC that enables fine-grained isolation and an integrated GPU. This includes missing and limited documentation for provided TZASC components. In this thesis, we therefore choose to work with a simulated ARM SoC containing an integrated GPU, a TZASC, and the capability to assign a distinct NSAID for each core on the bus. We discuss ARM Fast Models, the approach we use in the following chapters, in [Section 2.6](#).

2.6 ARM FAST MODELS

Fast Models [8] are programmer view models of processors and devices that work at a level where functional behavior is equivalent to what a programmer would see with hardware. They sacrifice timing accuracy to achieve fast simulation execution speeds. Thus, Fast Models can be used for confirming software functionality, but are not cycle accurate by default. Plug-ins are available for Fast Models that can be used for tracing model execution. We build a custom plug-in for providing instruction traces (see [Chapter 4](#)).

Fixed Virtual Platforms (FVPs) [7] are pre-built executables that are built by composing a system consisting of fast models. Their composition is fixed, but can have run-time options configured. FVPs can be used to simulate an entire Linux system with a shell prompt, including a TEE with ATF and OP-TEE [27]. Several pre-build FVPs exist and are provided by ARM.

We use the FVP_Base_RevC-2xAEMvA FVP as a base, detailed in [Section 2.6.1](#), in [Chapter 4](#). We separately discuss GPU fast models in [Section 2.6.2](#) due to their complexities and limitations. We discuss FVP simulation time, performance options, and performance evaluations methods in [Section 2.6.3](#).

2.6.1 FVP_Base_RevC-2xAEMvA Overview

FVP_Base_RevC-2xAEMvA by default uses the ARMv8 architecture, containing two clusters each consisting of four cores. Clusters can be configured to use different NSAsIDs. FVP_Base_RevC-2xAEMvA also includes the TZC-400 and the ARM Mali-G76 GPU. [Figure 2.11](#) shows the main components in the Base Platform RevC FVP.

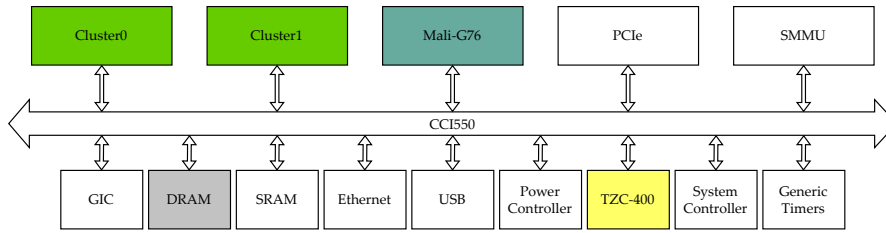


Figure 2.11: Base Platform RevC block diagram.

2.6.2 GPU Fast Models

GPU fast models do not provide a software renderer simulation of a real GPU. GPU fast models instead provide a register interface and only simulate interrupts. They run the full driver stack, allowing GPU device drivers and user space drivers to successfully interact with the fast model. However, the fast-models do not perform any computations and signal job completion immediately.

`libnomali.so` is the shared library responsible for simulating the GPU fast model interrupts and providing a register interface [6]. By default, `libnomali.so` only accesses registers and signals interrupts. No memory transactions are initiated by the fast model. However, the callback structure, shown in Listing 2.4, includes function pointers `memread` and `memwrite` which initiate memory transactions for reading and writing data, respectively. The function pointers are not used by default. Figure 2.12 shows the architectural location of `libnomali.so` within FVP_Base_RevC-2xAEMvA. We discuss `libnomali.so` extensions to simulate memory transactions in Chapter 4.

```

/**
 * Callback information structure.
 */
typedef struct {
    /** Callback type */
    nomali_callback_type_t type;
    /** Pointer to user-defined data associated with callback */
    void *usr;
    /** Pointer to callback function */
    union {
        /**
         * Interrupt state change
         *
         * @param h Model instance handle.
         * @param usr User-defined data associated with callback.
         * @param intno Interrupt number.
         * @param set Non-zero if raising an interrupt, zero if
         *          clearing.
         */
        void (*interrupt)(nomali_handle_t h, void *usr,
                        nomali_int_t intno, int set);
        void (*memwrite)(nomali_handle_t h, void *usr,
                        nomali_addr_t addr, uint32_t value);
        uint32_t (*memread)(nomali_handle_t h, void *usr,
                        nomali_addr_t addr);
        void (*reset)(nomali_handle_t h, void *usr);
    } func;
} nomali_callback_t;

```

Listing 2.4: libnomali.so callback information structure.

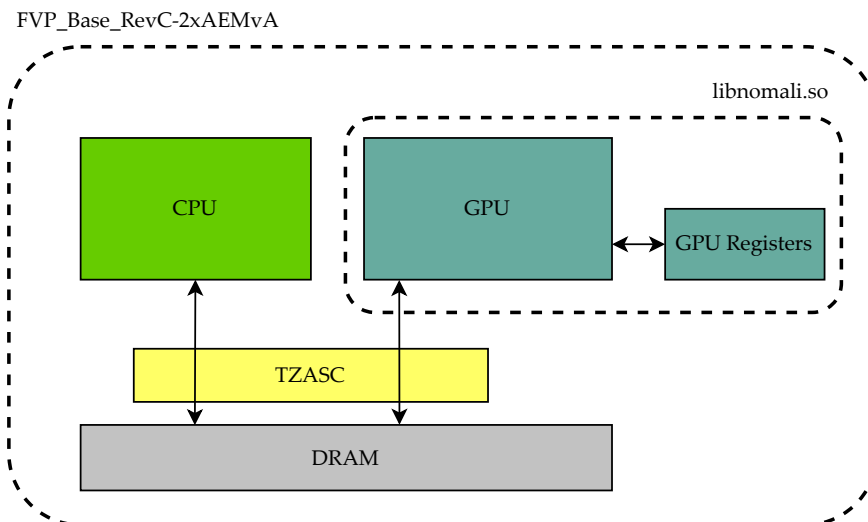


Figure 2.12: libnomali.so architectural location within FVP_Base_RevC-2xAEMvA.

2.6.3 FVP Timing & Performance Evaluation

Application run-times reported within a FVP simulation displays the simulated time. The simulated time is an estimate of the time the workload would take on the modeled hardware. For example, [Listing 2.5](#) runs `time sleep 5` within a running simulation. The real time reported is the simulated time. The time in wall clock seconds that the host CPU spent running this part of the simulation can vary greatly. Although fast models are not cycle accurate, we can evaluate and compare application performance by using the reported simulated time.

```
# time sleep 5
real    0m 5.01s
user    0m 0.00s
sys     0m 0.00s
```

Listing 2.5: FVP simulated time example.

Several options are available to adjust the cycle accuracy of fast models. Plug-ins `BanchPrediction` and `PipelineModel` can be included to model branch prediction and pipeline cycle delays, respectively. A Cycles Per Instruction (CPI) file can also be included to specify CPI values for specific instructions or groups of instructions. By default, all instructions have a CPI value of 1.0. [Listing 2.6](#) shows an example CPI file, which assigns `ADD` instructions a CPI value of 0.5 and all other instructions a CPI value of 0.75. Refer to [\[10\]](#) for further details on CPI files and FVP timing annotations.

```

# -----
# Instruction classes
# -----
DefineClass ADD_ext_reg Mask=0x7FE00000 Value=0x0B200000 ISet=A64
DefineClass ADD_sft_reg Mask=0x7F200000 Value=0x0B000000 ISet=A64
DefineClass ADD_imm      Mask=0x7F000000 Value=0x11000000 ISet=A64

# -----
# Instruction groups
# -----
DefineGroup ADD_instr Classes=ADD_ext_reg,ADD_sft_reg,ADD_imm
                    ISet=A64

# -----
# CPI values
# -----
DefineCpi ADD_instr ISet=A64 Cpi=0.50

# -----
# Defaults
# -----
Defaults ISet=* Cpi=0.75

```

Listing 2.6: FVP CPI example file.

DESIGN & ARCHITECTURE

We describe a TEE design for GPU-accelerated applications on ARM edge devices with integrated GPUs. We follow the Sanctuary model of isolation where we provide an isolation environment built in NW [15]. In addition to the Sanctuary model, we provide secure request-level GPU access within the TEE. We discuss design goals of our system in [Section 3.1](#). We discuss the threat model in [Section 3.2](#). We finally provide a design overview in [Section 3.3](#).

3.1 DESIGN GOALS

We aim to provide a TEE on ARM-based systems that supports secure integrated GPU acceleration by protecting data confidentially throughout the application execution. This includes protecting data accessed by the GPU. Any intermediate data generated during the execution must also be confidential. We aim to support concurrent GPU requests with per-request isolation and aim to support multi-tenancy. We aim to incorporate existing applications into the TEE with minimal application changes. We additionally aim to provide code (ML model) integrity. We finally aim to achieve the above goals by utilizing a minimal TCB with acceptable latency overheads.

3.2 THREAT MODEL

We consider a strong adversary that can comprise the OS in order to intrude and modify an application's execution. We therefore do not trust NAs and the NOS. Untrusted applications may be executing

simultaneously on the CPU and GPU. An adversary may attempt to violate confidentiality and integrity guarantees of the TEE through direct attacks. We assume the hardware SoC package is trusted. An adversary may not compromise the SoC package, consisting of cores, on-chip memory, TZASC, and the integrated GPU. An adversary may however attempt invasive physical attacks that tamper with off-core components, including peripherals, and may snoop on the PCIe interconnect. We assume the secure monitor (ATF) and SW software components, including the TOS (OP-TEE core) and TAs, are trusted. We do not consider denial of service attacks or side-channel attacks.

3.3 DESIGN OVERVIEW

We design a TEE for ARMv8-based devices with TrustZone security extensions and integrated GPUs. We assume presence of a First-Level Cache (L1) and a Second-Level Cache (L2) where L1 caches are core-exclusive while L2 caches are shared. Within the design, we assume that ARM cores can be assigned unique NSAIDs. This is not generally true in current available commercial hardware (see [Section 2.5](#) for details) which instead assign NSAIDs at the granularity of clusters. This does not however affect the design as the isolated cores can be assigned to separate clusters. [Figure 3.1](#) illustrates the setup of each world and the software component execution levels in each world.

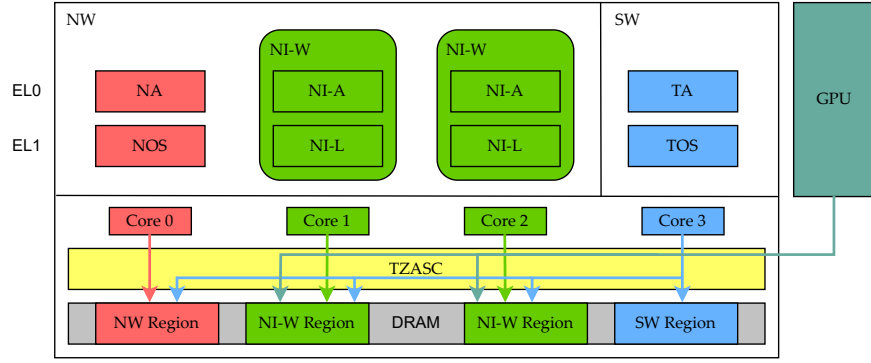


Figure 3.1: Design overview. Two NI-W instances are showcased. The diagram shows contiguous NI-W DRAM regions. This may not be the case and is a simplification of the diagram. Allowed memory accesses are shown with an arrow.

We define each software component below.

NA. A simple application that initiates execution of a GPU-accelerated application.

NI-W. Normal-Isolated World (NI-W). A separate world, isolated from NW.

NOS.KM. NOS Kernel Module (KM). Communicates with RA to setup the NI-W.

NI-A. Normal-Isolated Application (NI-A). GPU-accelerated application. Runs in EL0.

NI-L. Normal-Isolated Library (NI-L). Library containing the GPU device driver and basic functionality required for memory management between the CPU and GPU. We aim to keep the size of NI-L to a minimum by including only the necessary features. Runs in EL1.

RA. Remote Attestation Application (RA). SW application providing remote attestation and sealing functionality. RA also provides secure initialization of NI-W. RA consists of two components, RA.EL0 and RA.EL1. RA.EL0 is a TA implementing remote attestation and sealing. RA.EL0 runs in EL0. RA.EL1 is a TOS KM responsible for NI-W initialization. RA.EL1 runs in EL1.

- [NA, NI-A]. Shared memory region between NA and NI-A. Used to transfer sealed client data to a NI-W instance.
- [NI-A, RA]. Shared memory region between NI-A and RA. Used by NI-A when invoking RA services provided by either RA.EL0 or RA.EL1.
- [NI-A, GPU]. Shared memory region between NI-A and GPU used during GPU execution. NI-A sends client data to this region and reads GPU results from the region. GPU also accesses and writes results to the region. We require this region because integrated GPUs can only access a certain block of physical system memory (see [Section 2.4.3](#) for details).
- [NI-A]. Private memory region used by NI-W components. NI-A binary, NI-L binary, and dynamically allocated NI-W internal memory are allocated here.

The goal of the TEE architecture is to allow creation of multiple isolated NW compartments, NI-Ws, which are isolated from NAs and the NOS. We ensure isolation of NI-W instances by leveraging the TZASC to partition memory regions.

A NA initiates execution of a GPU-accelerated application by invoking setup of a NI-W. The GPU-accelerated application runs within NI-W after world setup. NI-W setup begins by running a NI-W handler process on a separate core from NA that will call NOS.KM to initialize and start a NI-A within NI-W. Therefore, NA and NI-A run on separate cores, which is essential to obtain core-based TZASC security guarantees.

A NI-W is created on a specific core distinct from NA and remains on the same core throughout NI-A execution. Context switching that occurs with SMC calls during setup and teardown to either NW or SW also occur on the same core. During NI-W initialization, interrupts from the system-wide interrupt controller, triggered from other cores and targeting the NI-W core, are disabled. Therefore, NI-A execution

cannot be preempted by another core. Only NI-A can teardown NI-W by issuing a teardown request to RA. Every GPU-accelerated application is mapped to a distinct NI-W instance.

We handle NI-W partitioning for a specific memory region separately for the application core and the GPU. The cores are configured by TZASC such that only the NI-W core can access the memory region. GPUs require special handling as normally a GPU presents to the system as a single bus master. We propose a simple hardware extension to enable multiple GPU NSAIDs such that a unique NSAID is used for a single NI-W instance. We implement the multiple NSAID extension for the GPU on FVPs in [Chapter 4](#). RA.EL1 contains a pool of available GPU NSAIDs and selects an available one when booting a NI-W instance. The trusted GPU device driver within NI-L is responsible for signaling to the GPU to use the provided GPU NSAID when invoking commands.

We require four separate memory regions to setup a NI-W instance. The regions are allocated by RA during NI-W initialization. All regions except for [NA, NI-A] are protected by TZASC configuration, done by RA.EL1 during NI-W initialization. [NA, NI-A] is only used for data communication between NA and NI-A and the data is sealed using a certificate provided by RA. For the TZASC configured regions, the GPU is only granted access to [NI-A, GPU]. TZASC is configured such that [NI-A, GPU] can be accessed by both the running NI-W core NSAID and the selected GPU NSAID. [NI-A, RA] and [NI-A] are accessible by the running NI-W core NSAID only. A running NI-W instance is only granted memory access to the four memory regions mentioned previously. This prevents an adversarial NI-W instance from attacking NAs. Similarly, the GPU NSAID is only granted memory access to [NI-A, GPU]. For the TZASC configured regions, the TOS and TAs can only access [NI-A, RA]. [Figure 3.2](#) illustrates the outcome of memory region partitioning, where allowed memory accesses are shown with an arrow.

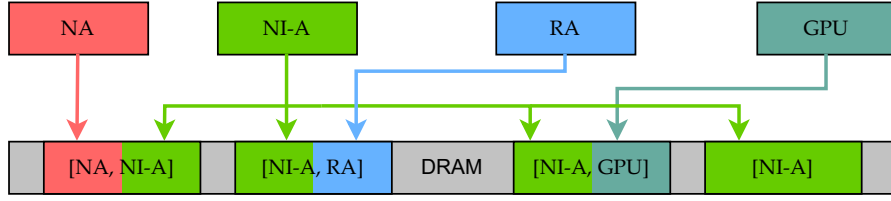


Figure 3.2: Memory region partitioning of NI-W memory regions during NI-W setup. Allowed memory accesses are shown with an arrow. The memory regions do not need to be contiguous and as a result we provide spacing between memory blocks.

The TCB consists of the following components.

- NI-A.
- NI-L.
- Secure monitor (ATF).
- TOS (OP-TEE).
- RA, including RA.EL0 and RA.EL1.

In the following, we describe in detail the steps for setting up a GPU-accelerated application within NI-W.

- S1. Secure boot and remote attestation of the platform. An initial boot into the TOS by the secure monitor followed by the TOS platform setup. After SW platform setup, the TOS context switches to NW for NOS boot and initialization.
- S2. Load both components of RA in SW, namely RA.EL0 and RA.EL1. RA.EL0 is loaded once during system boot and persists throughout the lifetime of the system. RA.EL1 is integrated with the TOS kernel. NAs see a single RA application. Which RA component invoked depends on the commands sent to the RA.
- S3. NA initiates execution of NI-A. A new NI-W handler process starts up on a distinct core that handles the remaining communication tasks involved in setting up a NI-W instance. This process invokes NOS.KM which initiates communication with RA.

- S4. RA.EL1 allocates memory regions [NA, NI-A], [NI-A, RA], [NI-A, GPU], and [NI-A]. The regions [NA, NI-A] and [NI-A] are communicated with NOS.KM. The region [NA, NI-A] is communicated with NA.
- S5. NOS.KM loads NI-A and NI-L sent from NA in [NI-A]. NI-L is signed and will have the signature verified in the following step. NOS.KM signals to RA to proceed with NI-W initialization. Context switching to NW does not occur again until NI-W teardown.
- S6. RA performs the following security functions.
- RA.EL1. Select an available GPU NSAID from the pool.
 - RA.EL1. Use TZASC to configure access permissions to memory regions [NI-A, RA], [NI-A, GPU], and [NI-A]. [NI-A, GPU] can only be accessed in NW by the NI-W core NSAID and the selected GPU NSAID. [NI-A, RA] and [NI-A] can only be accessed in NW by the NI-W core NSAID. The NI-W core NSAID and selected GPU NSAID can access no other memory regions. The TOS and TAs can only access [NI-A, RA].
 - RA.EL1. Zero-initialize memory regions, retaining the existing binaries.
 - RA.EL0. Compute hash of NI-A binary and NI-W configuration. Derive a key (NI-A key) based on this hash and sign the hash + NI-A key using a platform key.
 - RA. Send the certificate to NA using [NA, NI-A].
- S7. The client verifies the NI-A certificate, uses the NI-A key from it to seal confidential data and uploads the sealed data to NA.
- S8. NA loads the sealed data into [NA, NI-A].
- S9. RA.EL1 context switches into NI-W using SMC to begin execution of NI-L. The secure monitor is invoked which handles jumping to the start of NI-L to begin execution.

S10. NI-L begins execution and launches NI-A.

NI-A. Unseals data by communicating with RA.EL0.

NI-A. Allocates memory buffers for GPU in [NI-A, GPU] and copies data into the buffers. Invokes GPU via the NI-L GPU device driver.

GPU. Performs computations using the allocated memory buffers and returns results to NI-A in [NI-A, GPU].

NI-A. Returns sealed results to NA in [NA, NI-A].

S11. NI-A triggers teardown of NI-W after execution completes.

NI-A. Saves state using RA.EL0's sealing service.

NI-L. Restored to its original state.

S12. NI-A invokes RA using SMC to perform the final security services and return to NW.

S13. RA performs the following security functions.

RA.EL1. Zeroes memory regions.

RA.EL1. Caches invalidated.

RA.EL1. Removes TZASC configurations.

RA.EL1. Context switches into NW using SMC.

IMPLEMENTATION

We detail the necessary software changes and minimal hardware changes required to support a full implementation of the TEE design mentioned in [Chapter 3](#). Hardware extensions needed to support both the full TEE implementation and perform systematic evaluation of overheads present in the design are mentioned in [Section 4.1](#). Software extensions necessary to support both the full TEE implementation and perform systematic evaluation of overheads present in the design are mentioned in [Section 4.2](#). Finally, we discuss applications we create for TEE evaluation in [Section 4.3](#).

We target ARM fast models, building on top of the OP-TEE provided FVP solution consisting of Linux, ATF, and OP-TEE SW components [28]. We use platform version 3.15.0, corresponding to the following manifest file <https://github.com/OP-TEE/manifest/blob/3.15.0/fvp.xml>.

4.1 HARDWARE SIMULATION IMPLEMENTATION

By default, the OP-TEE platform uses the Foundation_Platformpkg FVP. Foundation_Platformpkg is limited in functionality and does not include the TZASC [8]. We therefore use FVP_Base_RevC-2xAEMvA, discussed further in [Section 2.6.1](#). As discussed in [Section 2.6.2](#), GPU fast models do not provide a software renderer simulation of a real GPU. To evaluate GPU memory accesses to ensure the GPU can only access permitted regions, configured by TZASC, we extend libnomali.so to simulate memory accesses. We additionally require partitioning the GPU to use a specific NSAID, as specified by the GPU device driver described in [Sec-](#)

tion 3.3. We therefore extend `libnomali.so` and `LabellerForGPUProtMode` (explained below) to support configuring the GPU to use specific NSAIDs during memory accesses.

We implement three hardware extensions in the simulation: (1) support for setting a distinct NSAID label for each core rather than a cluster (Section 4.1.1), (2) the simulation of memory operations of a Mali GPU and the setting of a NSAID value for each model instance executing on the GPU (Section 4.1.2), and (3) An instruction tracer that logs the sequence of CPU instructions executed, which we use subsequently for performance benchmarking (Section 4.1.3). We elaborate on these extensions in the rest of this section.

4.1.1 NSAID Core Granularity Extensions

By default, `FVP_Base_RevC-2xAEMvA` accepts command line arguments to configure NSAIDs for CPU clusters. The configuration parameters are shown in Listing 4.1, where we specify `cluster0` to have a NSAID of `0x0` and `cluster1` to have a NSAID of `0x1`.

```
-C cluster0_labeller.label=0x0
-C cluster1_labeller.label=0x1
```

Listing 4.1: Default `FVP_Base_RevC-2xAEMvA` NSAID configuration parameters.

We create a new fast model component—`CPUlabeller`—to set memory access NSAIDs at a core granularity. The `CPUlabeller` component is shown in Figure 4.1. The NSAID set by the component is based on the core that initiated the memory access. Each cluster uses a single `CPUlabeller` component. The process of setting the NSAID based on the core initiating the access is shown in Listing 4.2.

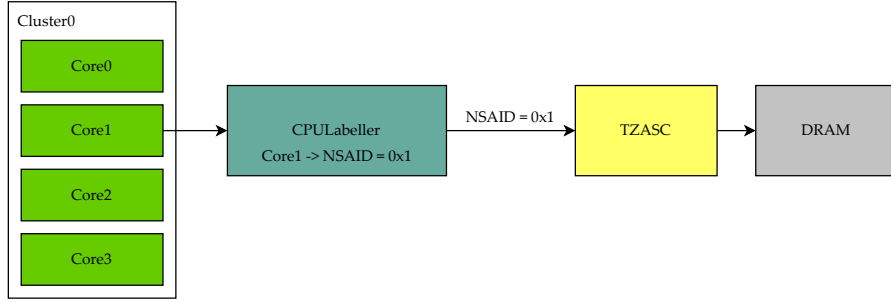


Figure 4.1: CPUlabeller fast model functionality. CPUlabeller intercepts CPU core memory transactions to assign an appropriate NSAID.

```

uint8_t NSAID_core[4] = {core0, core1, core2, core3};
uint8_t NSAID = NSAID_core[core_id];

setNSAID(NSAID);
  
```

Listing 4.2: CPUlabeller assigning core NSAIDs.

We specify core NSAIDs as command line arguments. The configuration parameters are shown in [Listing 4.3](#), where we assign NSAID values sequentially.

```

-C cluster0_labeller.core0=0
-C cluster0_labeller.core1=1
-C cluster0_labeller.core2=2
-C cluster0_labeller.core3=3

-C cluster1_labeller.core0=4
-C cluster1_labeller.core1=5
-C cluster1_labeller.core2=6
-C cluster1_labeller.core3=7
  
```

Listing 4.3: Extended FVP_Base_RevC-2xAEMvA NSAID configuration parameters, where we configure CPU NSAIDs at a core granularity.

4.1.2 GPU Fast Model Extensions

`libnomali.so` is invoked during GPU register accesses. We extend the register handling by adding the following three 32-bit registers, `SEND_ADDR_TO_GPU_LO`, `SEND_ADDR_TO_GPU_HI`, and `SEND_ADDR_TO_GPU_SIGNAL`. A 64-bit memory address is expressed using the combination of `SEND`

`_ADDR_TO_GPU_LO` and `SEND_ADDR_TO_GPU_HI`. `SEND_ADDR_TO_GPU_LO` represents the lower 32-bits of the address and `SEND_ADDR_TO_GPU_HI` represents the upper 32-bits of the address. Writing to `SEND_ADDR_TO_GPU_SIGNAL` must occur after writing to the address registers and triggers the GPU to perform a memory read and memory write to the specified address. [Listing 4.4](#) highlights the function `write_reg` of `libnomali.so`, which handles GPU register writes. Writing to `SEND_ADDR_TO_GPU_SIGNAL` triggers `handle_addr_to_gpu_signal` to perform a memory read and memory write using the `memread` and `memwrite` functions, respectively (discussed further in [Section 2.6.2](#)). The `memread` and `memwrite` function pointers are set during FVP boot. The source code for the set functions are not provided by ARM. To support configuring the GPU to use specific NSAIDs during memory accesses, we specify the NSAID to use in `SEND_ADDR_TO_GPU_SIGNAL` register writes. We then set the most significant nibble of the memory address to the NSAID. We extend the `LabellerForGPUProtMode` fast model component, shown in [Figure 4.2](#), to use the most significant nibble and set the GPU NSAID accordingly. The process of setting the GPU NSAID and remapping the base page address to remove the nibble is shown in [Listing 4.5](#).

```

void handle_addr_to_gpu_signal(uint8_t nsaid)
{
    nomali_addr_t callbackAddr;

    callbackAddr = (regs[SEND_ADDR_TO_GPU_HI] << 32) |
                  regs[SEND_ADDR_TO_GPU_LO];

    callbackAddr |= ((uint64_t) nsaid << 60);

    memread(callbackAddr);
    memwrite(callbackAddr);
}

void write_reg(RegAddr addr, uint32_t value)
{
    switch (addr) {
        // Additional registers removed for compactness
        case SEND_ADDR_TO_GPU_LO:
        case SEND_ADDR_TO_GPU_HI:
            regs[addr] = value;
            break;
        case SEND_ADDR_TO_GPU_SIGNAL:
            handle_addr_to_gpu_signal(value);
            break;
    };
}

```

Listing 4.4: libnomali.so register writing extensions.

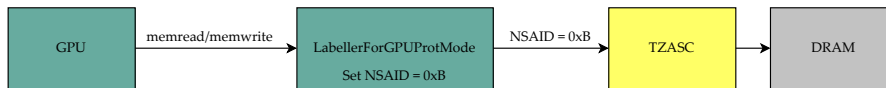


Figure 4.2: LabellerForGPUProtMode fast model functionality. LabellerForGPUProtMode intercepts GPU memory transactions to assign an appropriate NSAIID.

```

uint64_t addr = getPageBase();
uint8_t NSAIID = addr >> 60;

setPageBase(addr & 0xFFFFFFFFFFFFFFFF);
setNSAIID(NSAIID);

```

Listing 4.5: LabellerForGPUProtMode assigning GPU NSAIIDs.

4.1.3 *InstTrace*

We create a custom fast model plug-in *InstTrace* to trace instructions executed by a specific ARM core, for example, `FVP_Base_RevC_2xAEMvA.cluster1.cpu3`, which corresponds to the seventh CPU core presented by Linux. *InstTrace* reads `INST` trace sources provided by ARM cores. *InstTrace* periodically saves the instruction traces observed on the core to a file. A snapshot of the file is shown in [Listing 4.6](#). With some post-processing, we observe all the instructions executed by an application running on the traced core. Refer to [9] for more details regarding fast model plug-ins. *InstTrace* extends the *GenericTrace* plug-in from ARM Fast Models to provide the necessary functionality.

```

INST, COUNT
ADD, 1544847
ADDP, 4156
ADDS, 14479
ADR, 5831
ADRP, 261546
AND, 486002
ANDS, 28685
ASR, 35710
B, 513917
B.CC, 64738
B.CS, 40421
B.EQ, 448216
B.GE, 11594
B.GT, 12826
B.HI, 155176
B.LE, 24782
B.LS, 67581

```

Listing 4.6: *InstTrace* output file snapshot.

4.2 SOFTWARE IMPLEMENTATION

We implement support for dynamic configuration of the TZASC from the Linux kernel, utilizing the ATF in the process. OP-TEE already provides support for TZASC configuration. By default, the TZASC responds to permission violations by sending a Decode Bus Error (DECERR) [4].

This presents to the master as though the memory location requested is absent. We describe dynamic TZASC configuration and client failure detection in [Section 4.2.1](#).

We extend the Linux kernel and OP-TEE to support simulating GPU memory accesses in the simulation environment, using the hardware extensions mentioned in [Section 4.1](#). We describe Linux kernel and OP-TEE extensions to support simulating GPU memory accesses in [Section 4.2.2](#).

By default, memory accesses from either the OP-TEE core or from a TA to the physical shared memory region (discussed in [Section 2.2](#)) is treated as NS, with NS=1. We modify OP-TEE to instead use secure accesses, with NS=0. We additionally modify the TEEC_AllocateSharedMemory API to return the starting physical address of the allocated contiguous region for later TZASC configuration. We describe OP-TEE modifications to obtain NS=0 accesses and TEEC_AllocateSharedMemory API changes in [Section 4.2.3](#).

4.2.1 TZASC Dynamic Configuration & Fault Handling

We build a Linux character device driver, `tzc`, that handles dynamically configuring TZASC regions. The main entry point is a IOCTL command that invokes the `tzc_configure` function shown in [Listing 4.7](#). `tzc_configure` takes a `tzc_region` structure as an argument, shown in [Listing 4.7](#), that describes the TZASC region configuration options detailed in [Section 2.1.3](#). `tzc_configure` uses the Linux kernel provided function `arm_smccc_smc` to perform a SMC call that invokes ATF handling. We specify the SMC function ID `TZC_SMC_CALL_CONFIGURE`, which corresponds to our custom built `tzc` ATF service (Refer to [13] for more details regarding the SMC calling convention and SMC function IDs and to [31] for more details regarding ATF run-time services). The `tzc`

ATF service makes use of existing ATF TZASC configuration functions, namely `tzc400_configure_region`, to perform the remaining steps.

```
typedef struct {
    uint64_t filters;
    uint64_t region;
    uint64_t region_base;
    uint64_t region_top;
    uint64_t sec_attr;
    uint64_t nsaid_permissions;
} tzc_region;

long tzc_configure(unsigned long arg)
{
    union {
        struct arm_smccc_res smccc;
        tzc_configure result;
    } res;

    tzc_region region;

    copy_from_user(
        &region,
        (tzc_region *) arg,
        sizeof(tzc_region)
    );

    arm_smccc_smc(
        TZC_SMC_CALL_CONFIGURE,
        region.filters,
        region.region,
        region.region_base,
        region.region_top,
        region.sec_attr,
        region.nsaid_permissions,
        0,
        &res.smccc
    );

    return res.result.status;
}
```

Listing 4.7: tzc Linux device driver TZASC configuration handler.

The Linux kernel presents DECERRs to user processes through SIGBUS signals. We can therefore capture SIGBUS signals by invoking `sigaction` (`SIGBUS`, `&sa`, `NULL`). This is used by the empirical tests described in [Section 5.3.2](#). OP-TEE TAs that cause a DECERR are panicked during OP-TEE core fault handling. `TEEC_InvokeCommand` returns the error `TEEC`

`_ERROR_TARGET_DEAD`. This is used by the empirical tests described in [Section 5.3.4](#).

4.2.2 *GPU Memory Access Signaling*

We extend the Linux Mali GPU device driver to support an additional IOCTL command, `KBASE_IOCTL_SEND_ADDR_TO_GPU`. `kbase_send_addr_to_gpu`, shown in [Listing 4.8](#), handles the IOCTL command by writing to the registers we mention in [Section 4.1.2](#). We specify the memory address for the GPU to access and the NSAID for the GPU to use in the transactions with struct `kbase_ioctl_send_addr_to_gpu`, shown in [Listing 4.8](#).

```

struct kbase_ioctl_send_addr_to_gpu {
    uint64_t paddr;
    uint8_t nsaid;
};

static int kbase_send_addr_to_gpu(
    struct kbase_device *kbdev,
    struct kbase_ioctl_send_addr_to_gpu *send_addr_to_gpu
)
{
    kbase_reg_write(
        kbdev,
        MMU_REG(SEND_ADDR_TO_GPU_LO),
        send_addr_to_gpu->paddr & 0xFFFFFFFF
    );

    kbase_reg_write(
        kbdev,
        MMU_REG(SEND_ADDR_TO_GPU_HI),
        (send_addr_to_gpu->paddr >> 32) & 0xFFFFFFFF
    );

    // Signal GPU to read & write to paddr
    kbase_reg_write(
        kbdev,
        MMU_REG(SEND_ADDR_TO_GPU_SIGNAL),
        send_addr_to_gpu->nsaid
    );

    return 0;
}

```

Listing 4.8: Mali GPU Linux device driver GPU memory access handler.

For comparison, we also add minimal extensions to OP-TEE in the simulator to support GPU applications inside the SW as described below. Note that in practice, running a GPU application inside SW will require porting the full functionality of a GPU driver, which is non-trivial, especially with proprietary black-box drivers. Moreover, adding more code to SW is undesirable as it increases the size of the TCB with high privileges, thus increasing the risk of a compromise. We create a new TEE internal core API function `TEE_SendToGPU`. `TEE_SendToGPU` ultimately calls the function `do_send_to_gpu` that handles the GPU register writes. `do_send_to_gpu` is part of a custom built OP-TEE device driver. [Listing 4.9](#) shows the `TEE_SendToGPU` function prototype and the `do_send_to_gpu` implementation.

```

void TEE_SendToGPU(void *vaddr, uint8_t nsaid);

static TEE_Result do_send_to_gpu(void *vaddr, size_t nsaid)
{
    paddr_t paddr = virt_to_phys(vaddr);

    kbase_reg_write(
        MMU_REG(SEND_ADDR_TO_GPU_LO),
        paddr & 0xFFFFFFFF
    );

    kbase_reg_write(
        MMU_REG(SEND_ADDR_TO_GPU_HI),
        (paddr >> 32) & 0xFFFFFFFF
    );

    // Signal GPU to read & write to paddr
    kbase_reg_write(
        MMU_REG(SEND_ADDR_TO_GPU_SIGNAL),
        nsaid
    );

    return TEE_SUCCESS;
}

```

Listing 4.9: OP-TEE core GPU memory access handler.

4.2.3 Secure OP-TEE Accesses

Accessible memory regions are statically created during OP-TEE compilation using `register_phys_mem(type, addr, size)`. For example, `register_phys_mem(MEM_AREA_NSEC_SHM, TEE_SHMEM_START, TEE_SHMEM_SIZE)` registers the physical shared memory region to be within DRAM. The OP-TEE core uses the function `core_mmu_type_to_attr`, which among other things determines the access security of the registered memory regions. Adding `TEE_MATTR_SECURE` to the memory region attributes results in secure NS=0 memory accesses. OP-TEE TAs are handled similarly, where during TA initialization and virtual memory mapping we add `TEE_MATTR_SECURE` to the memory attributes. The TA initialization function trace is shown in [Listing 4.10](#), where we explicitly add `TEE_MATTR_SECURE`.

```

tee_ta_open_session
tee_ta_init_session
tee_ta_init_user_ta_session
ldelf_load_ldelf
vm_set_ctx
core_mmu_create_user_map
core_mmu_populate_user_map
TAILQ_FOREACH(r, &uctx->vm_info.regions, link)
    set_pg_region(dir_info, r, &pgt, &pg_info) {
        // Set region as secure
        struct tee_mmap_region r = {
            .va = region->va,
            .size = region->size,
            // Include 'TEE_MATTR_SECURE' to ensure
            // user-mode TA mapped regions are accessed with NS=0
            .attr = region->attr | TEE_MATTR_SECURE,
        };
    }

```

Listing 4.10: Sequence of functions called by OP-TEE core during TA initialization, highlighting the TEE_MATTR_SECURE addition.

We extend the API of `TEEC_AllocateSharedMemory` to additionally return the starting physical address of the contiguously allocated region. The physical address can then be used for TZASC configuration. The function prototype remains the same. We instead extend `TEEC_SharedMemory` to include the starting physical address. The function prototype and updated `TEEC_SharedMemory` are shown in [Listing 4.11](#). This is used by the empirical tests described in [Section 5.3.4](#).

```

TEEC_Result TEEC_AllocateSharedMemory(TEEC_Context* context,
                                       TEEC_SharedMemory* sharedMem);

typedef struct {
    void *buffer;
    size_t size;
    uint32_t flags;

    + // Starting physical address
    + uint64_t paddr;

    // Remaining implementation-defined fields omitted
} TEEC_SharedMemory;

```

Listing 4.11: `TEEC_AllocateSharedMemory` API changes.

4.3 APPLICATIONS FOR TEE EVALUATION

We discuss the implementation steps of four different applications we use during microbenchmark ([Section 5.1](#)) and macrobenchmark ([Section 5.2](#)) TEE evaluation. The applications compute the matrix multiplication of two square matrices A and B , storing the result in R . The applications differ in whether OpenCL is used with GPU utilization and whether the matrix computation step runs in NW or SW. The four applications are called `nw_cpu`, `nw_gpu`, `sw_cpu`, and `sw_gpu`.

`nw_cpu` and `sw_cpu` both execute the matrix multiplication code mentioned in [Listing 4.12](#). The difference is `sw_cpu` executes this loop in SW within a TA, where the arguments A , B , and R are passed as shared memory references. The TA function that executes the matrix multiplication is highlighted in [Listing 4.13](#).

```
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        for (k = 0; k < n; k++)
            R[i*m + j] += A[i*m + k] * B[k*m + j];
```

Listing 4.12: Matrix multiplication loop.

```

static TEE_Result matmul(uint32_t param_types,
                        TEE_Param params[4])
{
    uint32_t exp_param_types = TEE_PARAM_TYPES(
        TEE_PARAM_TYPE_VALUE_INPUT,
        TEE_PARAM_TYPE_MEMREF_INPUT,
        TEE_PARAM_TYPE_MEMREF_INPUT,
        TEE_PARAM_TYPE_MEMREF_INPUT
    );

    if (param_types != exp_param_types)
        return TEE_ERROR_BAD_PARAMETERS;

    uint32_t n = params[0].value.a;
    uint32_t m = params[0].value.a;

    uint32_t* A = params[1].memref.buffer;
    uint32_t* B = params[2].memref.buffer;
    uint32_t* R = params[3].memref.buffer;

    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            for (k = 0; k < n; k++)
                R[i*m + j] += A[i*m + k] * B[k*m + j];

    return TEE_SUCCESS;
}

```

Listing 4.13: TA matrix multiplication function.

nw_gpu and sw_gpu both use OpenCL for GPU-accelerated matrix multiplication. OpenCL initialization is done in NW for both applications and is shown in [Listing 4.14](#). nw_gpu and sw_gpu both additionally utilize the GPU memory access signaling extensions detailed in [Section 4.2.2](#). nw_gpu uses the KBASE_IOCTL_SEND_ADDR_TO_GPU IOCTL command. [Listing 4.15](#) shows the function nw_send_to_gpu, which we call to invoke the IOCTL command. sw_gpu calls the function TEE_SendToGPU within a TA to signal GPU memory accesses. [Listing 4.16](#) shows the wrapper function sw_send_to_gpu, which we use for invoking TEE_SendToGPU.

```

void openc1_init()
{
    int n, m;

    cl_platform_id platform;
    cl_device_id device;
    cl_context ctx;
    cl_program program;
    cl_command_queue queue;

    cl_context_properties props[3] =
        { CL_CONTEXT_PLATFORM, 0, 0 };

    cl_kernel k_matrix_mul;

    clGetPlatformIDs(1, &platform, NULL);
    clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1,
        &device, NULL);

    // Create OpenCL context
    ctx = clCreateContext(props, 1, &device, NULL, NULL, NULL);

    // Build kernel program
    program = clCreateProgramWithSource(ctx, 1,
        (const char **) &kernel_code, NULL, NULL);
    clBuildProgram(program, 0, NULL, NULL, NULL, NULL);

    // Create input and output buffers
    A = clCreateBuffer(ctx, CL_MEM_ALLOC_HOST_PTR,
        n * m * sizeof(int), NULL, NULL);
    B = clCreateBuffer(ctx, CL_MEM_ALLOC_HOST_PTR,
        n * m * sizeof(int), NULL, NULL);
    R = clCreateBuffer(ctx, CL_MEM_ALLOC_HOST_PTR,
        n * m * sizeof(int), NULL, NULL);

    // Create a command queue for submitting commands
    queue = clCreateCommandQueue(ctx, device, 0, NULL);

    // Create kernel object
    k_matrix_mul = clCreateKernel(program, "matrix_mul", NULL);

    // Add arguments to the kernel
    clSetKernelArg(k_matrix_mul, 0, sizeof(int), &m);
    clSetKernelArg(k_matrix_mul, 1, sizeof(int), &n);
    clSetKernelArg(k_matrix_mul, 2, sizeof(cl_mem), &A);
    clSetKernelArg(k_matrix_mul, 3, sizeof(cl_mem), &B);
    clSetKernelArg(k_matrix_mul, 4, sizeof(cl_mem), &R);
}

```

Listing 4.14: nw_gpu and sw_gpu OpenCL initialization. Error checking is removed.

```

void nw_send_to_gpu(void *vaddr)
{
    struct kbase_ioctl_send_addr_to_gpu send_addr_to_gpu;

    int fd      = open("/dev/malio", 0_RDWR);
    uint64_t paddr = to_physical((uint64_t) vaddr);

    send_addr_to_gpu.paddr = paddr;
    send_addr_to_gpu.nsaid = FVP_NSAID_GPU;

    ioctl(fd, KBASE_IOCTL_SEND_ADDR_TO_GPU, &send_addr_to_gpu);

    close(fd);
}

```

Listing 4.15: GPU memory access signaling function `nw_send_to_gpu`.

```

void sw_send_to_gpu(void *vaddr)
{
    TEE_SendToGPU(vaddr, FVP_NSAID_GPU);
}

```

Listing 4.16: GPU memory access signaling function `sw_send_to_gpu`.

We discuss the implementation steps for each application below.

nw_cpu: This application computes the matrix multiplication on the CPU in NW. The steps of the test are explained in [Table 4.1](#).

STEP	WORLD	DETAILS
1	NW	buffer allocation
2	NW	buffer initialization
3	NW	matrix multiplication execution

Table 4.1: `nw_cpu` application steps.

nw_gpu: This application computes the matrix multiplication on the GPU in NW. All steps except for Mali GPU execution are performed. GPU execution latency is calculated based off the `nw_cpu` test. The steps of the test are explained in [Table 4.2](#).

STEP	WORLD	DETAILS
1	NW	buffer allocation
2	NW	buffer initialization
3	NW	OpenCL initialization
4	NW	GPU data/kernel transfer
5	N/A	GPU execution (sw_cpu{matmul_execute} / 100)

Table 4.2: nw_gpu application steps.

sw_cpu: This application computes the matrix multiplication on the CPU in SW. The steps of the test are explained in [Table 4.3](#).

STEP	WORLD	DETAILS
1	NW	TEEC session initialization
2	NW	buffer allocation
3	NW	buffer initialization
4	NW	TEEC invoke TA
5	SW	matrix multiplication execution

Table 4.3: sw_cpu application steps.

sw_gpu: This application computes the matrix multiplication on the GPU in SW. All steps except for Mali GPU execution are performed. GPU execution latency is calculated based off the sw_cpu test. The steps of the test are explained in [Table 4.4](#).

STEP	WORLD	DETAILS
1	NW	TEEC session initialization
2	NW	buffer allocation
3	NW	buffer initialization
4	NW	OpenCL initialization
5	NW	TEEC invoke TA
6	SW	GPU data/kernel transfer
7	N/A	GPU execution (<code>sw_cpu{matmul_execute}</code> / 100)
8	NW	TEEC session close

Table 4.4: `sw_gpu` application steps.

EVALUATION

We evaluate specific overheads that will be encountered in the full TEE implementation described in [Chapter 3](#). This includes overheads present in world switches, memory region partitioning using TZASC, and secure GPU handling. We construct various scenarios to compare and evaluate the overheads present, which will directly resemble overheads present in particular steps of the full TEE implementation. We additionally evaluate the security of the TEE by providing both a theoretical analysis and an empirical analysis. The empirical analysis consists of various scenarios to ensure aspects of the TEE design provide the necessary security guarantees. We conduct our testing using the modified FVP_Base_RevC-2xAEMvA FVP, discussed in [Section 2.6.1](#) and extended in [Section 4.1.2](#). We discuss microbenchmarks in [Section 5.1](#) and macrobenchmarks in [Section 5.2](#). We provide a theoretical and empirical security analysis of the TEE design in [Section 5.3](#).

5.1 MICROBENCHMARKS

Since Fast Models is not a cycle accurate simulator, it is difficult to obtain realistic performance measurements in terms of latencies. Therefore, as a proxy, we first measure the performance of applications in terms of the number of instructions executed as part of the application runtime. We compare the instruction count when the application is run on an unmodified NA in NW (i.e., without any security guarantees) and when it is run as a TA in SW. Note that since our TEE is built with similar hardware configurations for TZASC, The performance profile for NI-A will be similar to that of an NA. Therefore, we omit the results

for NI-A for brevity.

For our microbenchmark testing, we obtain instruction counts for specific parts of an application. We use `InstTrace`, discussed in [Section 4.1.3](#), to obtain instruction counts.

We compare NAs to there TA alternative by evaluating the instruction count differences and overheads present when setting up and invoking TAs. We evaluate the instruction count differences between NAs and TAs in [Section 5.1.1](#). We evaluate TA overheads in [Section 5.1.2](#).

5.1.1 *Differences in NW & SW Instruction Counts*

We analyze instruction count differences observed when executing an application in SW as opposed to executing the application in NW. We run two applications, `nw_cpu` and `sw_cpu`, explained in [Section 4.3](#). We use the custom fast model trace component `InstTrace` to obtain the instruction counts.

Overall, the instruction count increase in `sw_cpu` is minimal. `sw_cpu` executes 10% more instructions than `nw_cpu`. [Table 5.1](#) shows the instruction count percentage increase in `sw_cpu` per instruction group and instruction group weight, which denotes the fraction of the total `sw_cpu` instructions the instruction group represents. The large exception class percent increase is due to the SMC calls executed in `sw_cpu`. However, the group weight is negligible, since only a few SMCs are invoked. Thus, we expect that the overall overhead of the SMCs on the application latencies will be small. The remaining instruction groups have higher weight but smaller overheads. We note that the results may have some inaccuracies since `InstTrace` cannot capture the instructions between the precise application start and end; rather it relies on starting (and stopping) the tracing with a sufficient gap

before (respectively after) starting the application execution. Refer to [3] for more information regarding the instructions included in each instruction group. Overall, the primary overhead for running an application as a TA is in initializing the application inside SW; subsequent execution of the application in SW has similar performance as in NW.

INSTRUCTION GROUP	WEIGHT	PERCENT INCREASE
Data Processing	0.480	8%
Sys	0.002	2%
Exception	4.874×10^{-7}	1800%
Branch	0.025	20%
Load	0.439	7%
Store	0.054	40%

Table 5.1: sw_cpu instruction count percent increase over nw_cpu per instruction group.

5.1.2 SW Instruction Overheads

We analyze instruction count overheads present when running TAs. The difference between a NA and a TA is the OP-TEE client API functions invoked from NW. We therefore run a TA that when invoked does nothing and simply returns. The main TA function called when a TA is invoked, namely `TA_InvokeCommandEntryPoint`, which normally calls additional functions depending on `cmd_id`, instead returns immediately. This function is shown in [Listing 5.1](#). Using this modified function we can determine the overheads present in both starting up and tearing down a TA.

```

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx,
                                     uint32_t cmd_id,
                                     uint32_t param_types,
                                     TEE_Param params[4])
{
    return TEE_SUCCESS;
}

```

Listing 5.1: TA_InvokeCommandEntryPoint immediately returns.

We measure the instruction counts per OP-TEE client API function. The log transformed instruction counts for each function are shown in [Figure 5.1](#). The overhead is largely attributed to TEEC_OpenSession which executes a total of 26540983 instructions that accounts for 96% of the total number of overhead instructions. This is due to the number of tasks accomplished by TEEC_OpenSession, including TA binary loading from NW file system storage into NS shared memory, TA binary loading into secure memory, and preparing the TA for execution. The NW OP-TEE driver and SW OP-TEE core are involved in several communication rounds to facilitate these tasks. This is highlighted in [Table 5.2](#), where TEEC_OpenSession executes a total of 72 SMC calls, corresponding to 36 communication rounds.

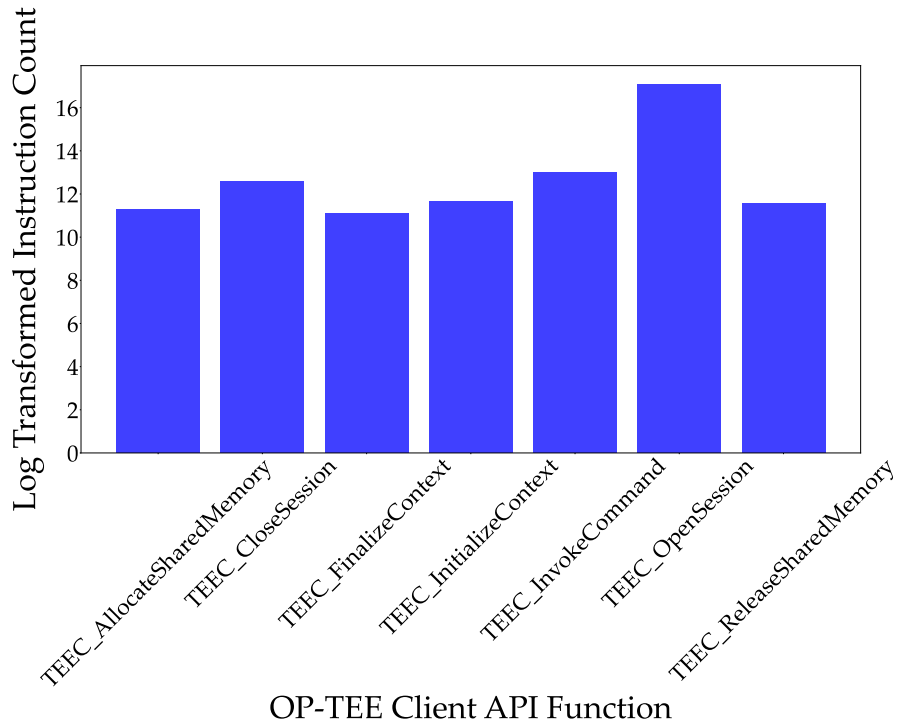


Figure 5.1: OP-TEE client API log base-10 transformed instruction counts.

OP-TEE CLIENT API FUNCTION	SMC COUNT
TEEC_AllocateSharedMemory	0
TEEC_CloseSession	2
TEEC_FinalizeContext	0
TEEC_InitializeContext	0
TEEC_InvokeCommand	2
TEEC_OpenSession	72
TEEC_ReleaseSharedMemory	0

Table 5.2: OP-TEE client API function SMC calls.

5.2 MACROBENCHMARKS

We measure application latency using FVP simulated time, which is an estimate of the time the application would take on the modeled hardware. Refer to [Section 2.6.3](#) for more details on FVP timing analysis. For simplicity, we use default CPI values for every instruction, which

is 1.0. A future work can repeat the relevant testing we describe in the following sections using a custom CPI file, described in [Section 2.6.3](#), for a more realistic measure of simulated time. However, creating a custom CPI file is difficult and our approach of separately obtaining instruction counts delivers accurate modeled hardware results.

We evaluate latency overhead present in OpenCL and SW applications. We use the simulated time provided by FVP (discussed further in [Section 2.6](#)) for determining the latency. We run four different applications, `nw_cpu`, `nw_gpu`, `sw_cpu`, and `sw_gpu`. The applications are explained in [Section 4.3](#).

We divide the latency for each application into four components, explained in [Table 5.3](#). Steps that do not apply for the application are assigned zero.

LATENCY COMPONENT	DETAILS
init	buffer allocation + buffer initialization + OpenCL initialization
to_sw	TEEC session init + TEEC invoke + TEEC session close
to_gpu	GPU data transfer + GPU kernel transfer
matmul_execute	Matrix multiplication execution.

Table 5.3: Latency components for `nw_cpu`, `nw_gpu`, `sw_cpu`, and `sw_gpu`.

We run each application on varying power-of-two square matrix sizes, ranging from 16x16 to 512x512. [Figure 5.2](#) shows the latency output for each application run, where we remove the `matmul_execute` step of the applications. We see a constant SW overhead. This is largely attributed to TA session initialization, with OP-TEE client API function calls `TEEC_InitializeContext` and `TEEC_OpenSession` (see [Section 2.3](#) for details). We additionally see a constant init overhead, where GPU applications have a larger overhead due to OpenCL initialization. OpenCL overhead

includes identifying GPU devices (`clGetDeviceIDs`), creating a context (`clCreateContext`), building the kernel program (`clBuildProgram`), creating buffer objects (`clCreateBuffer`), and creating the command queue (`clCreateCommandQueue`). For the GPU applications, we observe a positive correlation between matrix sizes and `to_gpu` latency. The `to_gpu` latency only becomes observable in the 128x128 case. Figure 5.3 includes the `matmul_execute` step. Matrix multiplication execution latency exponentially increases in the CPU applications and quickly becomes the dominating component of the overall latency.

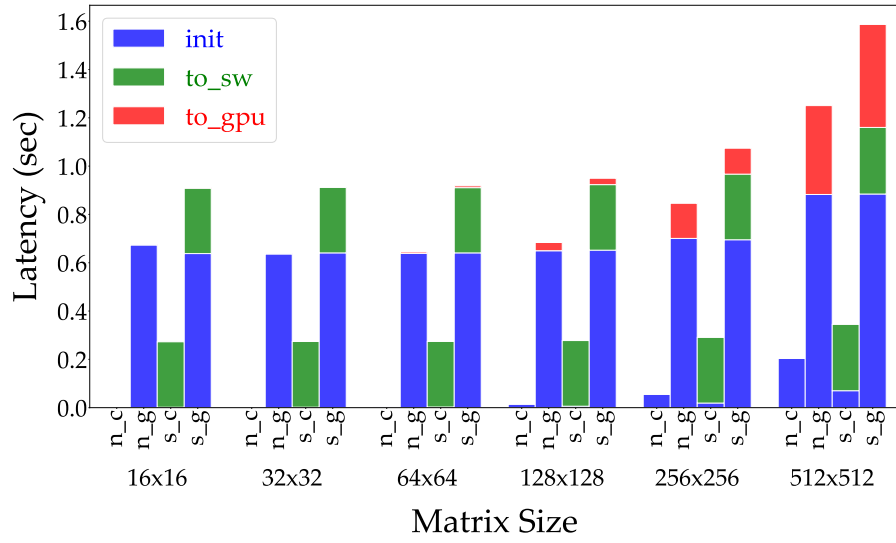


Figure 5.2: Application latency of `nw_cpu`, `nw_gpu`, `sw_cpu`, and `sw_gpu`. Matrix sizes ranging from 16x16 to 512x512 are included. The `matmul_execute` step is not included.

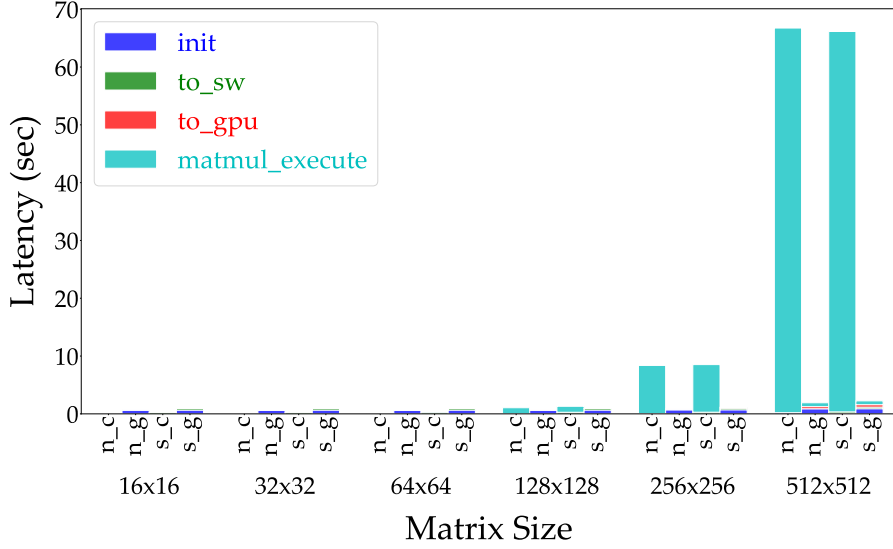


Figure 5.3: Application latency of nw_cpu, nw_gpu, sw_cpu, and sw_gpu. Matrix sizes ranging from 16x16 to 256x256 are included.

Looking at the overall latencies we see that the SW and OpenCL overheads result in the SW and GPU overall latencies being greater than the NW CPU application counterpart at smaller matrix sizes. As expected, an increase in the matrix sizes increases the multiplication cost exponentially on CPUs but only marginally on GPUs. Secondly, executing the application from SW incurs a fixed cost at the time of application initialization in the SW, which gets amortized over larger computations. Thirdly, the costs of data transfers to/from the GPU are proportional in the size of the transfers but constitute a very small percentage of the overall execution latency of the GPU-accelerated application. These results indicate that applications can be configured to use GPUs from within the TEE with low overheads, thus providing both security and efficiency.

5.3 SECURITY ANALYSIS

We evaluate the security of the TEE design by providing both a theoretical analysis and an empirical analysis. The theoretical analysis consists of considering various adversarial scenarios and evaluating the security guarantees provided by the design. The empirical analysis

consists of various TZASC configuration scenarios to ensure aspects of the TEE design provide the necessary security guarantees. We provide a theoretical analysis in [Section 5.3.1](#). We provide an empirical analysis of TZASC configuration in [Section 5.3.2](#), [Section 5.3.3](#), and [Section 5.3.4](#).

5.3.1 *Theoretical*

The goal of our design described in [Chapter 3](#) is to protect against a strong adversary. We discuss the theoretical security of our design by analysing all possible attack vectors based on our threat model, described in [Section 3.2](#). An attacker can originate from a NA, NOS, or malicious NI-A. The primary goal of an attacker is to compromise the data confidentiality and integrity of a NI-A. We discuss in the following paragraphs how our design ensures binary integrity, data isolation, and protection from malicious NI-As. We discuss how the security guarantees are maintained at all points in the NI-A lifecycle, namely during setup, execution, and teardown. We will show that an adversary cannot compromise the security provided by the TEE.

Binary integrity: Both NI-A and NI-L are unencrypted in the NW file system. During NI-W setup, we verify the integrity of NI-L by having its signature, which is signed using the platform key, verified by RA. If verification fails, NI-W setup is aborted. NI-A integrity is verified by remote attestation, where clients verify the received NI-A certificate. If verification fails, NI-W setup is aborted. Since both NI-A and NI-L integrity checking is done after TZASC configuration, only the running NI-W core can access the binary memory regions. After TZASC configuration, the NI-W core remains in either NI-W or SW during setup and execution. Note that each NI-A has its own NI-L instance and each NI-A/NI-L instance is either mapped to a separate core or executed on the same core as another instance after the previous instance has finished execution. Thus, NI-A/NI-L instances are spatially and/or tem-

porally isolated from each other. Therefore, an adversary in NW or NI-W of a different core cannot compromise the integrity of either NI-A or NI-L.

Data isolation: We use TZASC to partition memory regions [NI-A, RA], [NI-A, GPU], and [NI-A]. TZASC can only be configured from ATF or TOS, which are assumed to be trusted. [NI-A, GPU] can only be accessed in NW by the NI-W core NSAID and the selected GPU NSAID. The selected GPU NSAID is removed from the RA pool of GPU NSAIDs during NI-W setup and therefore the GPU NSAID is only used for a single NI-W instance. [NI-A, RA] and [NI-A] contain binary data and unsealed application data and can only be accessed by the NI-W core NSAID. After TZASC configuration, the NI-W core remains in either NI-W or SW during setup and execution. Therefore, an adversary in NW or NI-W cannot compromise the confidentiality and integrity of the TZASC configured regions. Additionally, client data uploaded into [NA, NI-A] is sealed and can only be unsealed through NI-A invoking RA services. The unsealed data is then stored in the TZASC configured region [NI-A, RA]. Therefore, an adversary cannot compromise the confidentiality and integrity of client provided data. Data saved during NI-A teardown is sealed by invoking RA services and consequently cannot be compromised by an adversary. Additionally, NI-A data is zero-initialized during teardown, which prevents any data leakage.

Protection from malicious NI-A's: Our current design allows potentially malicious NI-As from executing in NI-W. A malicious NI-A may attempt to compromise NW software, SW software, or separate NI-W software. A NI-A can only access memory regions [NA, NI-A], [NI-A, RA], [NI-A, GPU], and [NI-A]. These regions are explicitly set by RA and are never shared between NI-W instances. Therefore, a malicious NI-A cannot compromise software running in any world. A NI-A successfully performing a privilege-escalation attack by compromising NI-L does not affect the TZASC configuration guarantees. Since NI-L runs at

EL1 in NW, it cannot access SW memory and is still bound by the NW TZASC configuration for the running core. Additionally, a malicious NI-A cannot cause data leaks due to zero-initializing memory during NI-W setup and teardown.

5.3.2 *Empirical NW TZASC Configuration*

We evaluate general TZASC functionality by restricting memory region access to a single ARM cluster. We run two different NAs. The first application runs two threads where both threads attempt to access a shared memory region of page length. The first thread runs on cluster0 while the second one runs on cluster1. The cluster0 thread writes to the shared memory region while the cluster1 thread attempts to read from the shared memory region. [Listing 5.2](#) and [Listing 5.3](#) provide the relevant code sections of the application, where we have removed miscellaneous error checking. The second application is similar to the first, but instead runs two separate processes. The first process runs on cluster0 while the second one runs on cluster1. The cluster0 process writes to a memory region and saves the physical address of the region to a file. The second process obtains the physical address from the file and attempts to read from the physical region. Both applications can optionally use TZASC to configure the memory region to only allow NS access from cluster0. This is determined depending on the value of `configure_tzc`, shown in [Listing 5.2](#). We obtain identical results when running either application. We therefore only discuss the results of running the multithreaded application. The following paragraphs describe the outcome of both multithreaded application variants.

```

void main()
{
    char *vaddr = malloc(PAGE_SIZE);

    uint64_t region_base = to_physical(
        (uint64_t) vaddr & ~(PAGE_SIZE - 1)
    );

    uint64_t region_top = region_base + PAGE_SIZE - 1;

    if (configure_tzc) {
        tzc_region region = {
            .filters      = TZC_CLUSTER_FILTER,
            .region       = 2,
            .region_base  = region_base,
            .region_top   = region_top,
            .sec_attr     = 0,
            .nsaid_permissions =
                TZC_REGION_ACCESS_RDWR(FVP_NSAID_CLUSTER0)
        };

        open(TZC_DEV_NAME, O_RDWR);
        ioctl(fd, TZC_CONFIGURE, &region);
    };

    pthread_create(
        &thread1,
        NULL,
        cluster0_thread_handler,
        (void *) vaddr
    );

    pthread_create(
        &thread2,
        NULL,
        cluster1_thread_handler,
        (void *) vaddr
    );
}

```

Listing 5.2: Application for multithreaded general TZASC configuration, highlighting the main function.

```

void* cluster0_thread_handler(void *ptr)
{
    char *vaddr = (char *) ptr;

    char new_value = 110;

    set_thread_cpu_affinity(CLUSTER0_CORE);

    INFO("setting vaddr[o] to %d\n", new_value);
    vaddr[0] = new_value;

    return NULL;
}

void* cluster1_thread_handler(void *ptr)
{
    volatile char *vaddr = (char *) ptr;

    int sleep_time = 5;

    set_thread_cpu_affinity(CLUSTER1_CORE);

    INFO("sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);

    INFO("reading vaddr[o]\n");
    INFO("vaddr[o]=%d\n", vaddr[0]);

    return NULL;
}

```

Listing 5.3: Application for multithreaded general TZASC configuration, highlighting the thread handlers.

Multithreaded application with no TZASC configuration: The cluster1 thread is able to access the shared memory region. [Listing 5.4](#) shows the terminal output after the application terminates, where we indicate which thread the output corresponds to.

```

cluster0_thread_handler: setting vaddr[0] to 110
cluster0_thread_handler: reading vaddr[0]
cluster0_thread_handler: vaddr[0]=110
cluster1_thread_handler: sleeping for 5 seconds
cluster1_thread_handler: reading vaddr[0]
cluster1_thread_handler: vaddr[0]=110

```

Listing 5.4: Output of running the multithreaded application with no TZASC configuration.

Multithreaded application with TZASC configuration: The cluster1 thread is unable to access the shared memory region. The process obtains a SIGBUG signal when the cluster1 thread attempts to read from the shared memory region. [Listing 5.5](#) shows the terminal output after the application terminates, where we indicate which thread the output corresponds to.

```
cluster0_thread_handler: setting vaddr[0] to 110
cluster0_thread_handler: reading vaddr[0]
cluster0_thread_handler: vaddr[0]=110
cluster1_thread_handler: sleeping for 5 seconds
cluster1_thread_handler: reading vaddr[0]
Bus error si_code=BUS_OBJERR (Object-specific hardware error)
```

Listing 5.5: Output of running the multithreaded application with TZASC configuration.

5.3.3 Empirical GPU TZASC Configuration

We evaluate GPU TZASC configuration by optionally preventing GPU physical memory access to a specified region. We use the `nw_cpu` application as a base (described in [Section 5.2](#)) and extend it to optionally configure the buffer regions to prevent GPU access. The TZASC configuration is similar to [Section 5.3.2](#) where we additionally add the GPU NSAID to allow GPU access. This change is shown in [Listing 5.6](#). The following paragraphs describe GPU memory access outcomes, depending on TZASC configuration. We additionally discuss how GPU caches handle changes to TZASC configuration.

```

tzc_region region = {
    .filters          = TZC_CLUSTER_FILTER,
    .region           = 2,
    .region_base      = region_base,
    .region_top       = region_top,
    .sec_attr         = 0,
    .nsaid_permissions =
        TZC_REGION_ACCESS_RDWR(FVP_NSAID_GPU) |
        TZC_REGION_ACCESS_RDWR(FVP_NSAID_CLUSTER1)
};

```

Listing 5.6: TZASC GPU tzc_region configuration.

GPU access allowed: [Listing 5.7](#) shows the FVP logs corresponding to the GPU attempting to write to one of the buffers at physical address 0x89002E000. We see that the write passes TZASC checks and is allowed to occur.

```

FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xB
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xB) -> true

```

Listing 5.7: GPU write access allowed FVP logs.

GPU access prevented: [Listing 5.8](#) shows the FVP logs corresponding to the GPU attempting to write to one of the buffers at physical address 0x89002E000. We see that the write fails TZASC checks and does not occur.

```

FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xB
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xB) -> false
FVP: GPU model failed to write to 0x89002E000

```

Listing 5.8: GPU write access prevented FVP logs.

GPU cache handling: We first allow GPU write access, signal the GPU to perform multiple writes at physical address 0x89002E000, and then revoke GPU write access to observe future cache behavior. [Listing 5.9](#) shows the FVP logs corresponding to the steps described above. We annotate the listing to provide additional details as GPU cache accesses are not logged. We see that the cached page 0x89002E000 is invalidated after TZASC configuration changes, which results in future

writes failing. [Listing 5.10](#) additionally showcases that the cache is NSAID specific, such that changing the GPU NSAID results in skipping previously cached data.

```
# Configure TZASC to allow GPU write access

# First GPU write
FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xB
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xB) -> true

# Second GPU write uses the internal GPU cache

# Configure TZASC to prevent GPU write access

# GPU attempts to write again
FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xB
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xB) -> false
FVP: GPU model failed to write to 0x89002E000
```

Listing 5.9: GPU cache behavior during TZASC changes.

```
# GPU using NSAID=0xB
FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xB
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xB) -> true

# GPU using NSAID=0xC. Transaction completes fully
# and does not use previously cached data.
FVP: isNormalWorld=1 addr=0x89002E000 isRead=0 NSAID=0xC
FVP: tzc_400::validate(0x89002E000, write, NSAID=0xC) -> true
```

Listing 5.10: GPU cache behavior during NSAID changes.

5.3.4 Empirical SW TZASC Configuration

We evaluate SW TZASC configuration by optionally preventing SW physical memory access to a specified NS DRAM region. We create a NA that invokes a TA, passing a shared memory reference as an argument. We optionally configure TZASC to prevent SW access to this region, namely by setting `sec_attr` to 0, before invoking the TA. The NA is shown in [Listing 5.12](#) and the corresponding TA is shown in [Listing 5.11](#). We remove miscellaneous error checking in both listings. The follow-

ing paragraphs describe TA memory access outcomes, depending on TZASC configuration.

```
static TEE_Result handle_call(uint32_t param_types,
                             TEE_Param params[4])
{
    uint32_t exp_param_types =
        TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE);

    if (param_types != exp_param_types)
        return TEE_ERROR_BAD_PARAMETERS;

    uint8_t *buf = (uint8_t *) params[0].memref.buffer;

    DMSG("reading shm.buffer[o]");
    DMSG("shm.buffer[o]=%d", buf[0]);
    DMSG("writing shm.buffer[o] -> 110");

    buf[0] = 110;

    DMSG("shm.buffer[o]=%d", buf[0]);

    return TEE_SUCCESS;
}
```

Listing 5.11: TA attempting to access a shared memory region.

```

void call_ta(uint32_t configure_tzc)
{
    TEEC_InitializeContext(NULL, &ctx);
    TEEC_OpenSession(&ctx, &sess, &uuid,
        TEEC_LOGIN_PUBLIC, NULL, NULL, &err_origin);

    memset(&op, 0, sizeof(op));

    op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_WHOLE,
        TEEC_NONE, TEEC_NONE, TEEC_NONE);

    shm.size = page_size;
    shm.flags = TEEC_MEM_INPUT;

    TEEC_AllocateSharedMemory(&ctx, &shm);

    op.params[0].memref.parent = &shm;
    op.params[0].memref.size = shm.size;
    op.params[0].memref.offset = 0;

    ((uint8_t *) shm.buffer)[0] = 111;

    INFO("shm.buffer[o]=%d\n", ((uint8_t *) shm.buffer)[0]);

    if (configure_tzc) {
        region_base = shm.paddr & ~(page_size - 1);
        region_top = region_base + page_size - 1;

        // Prevent OP-TEE secure access to SHM memory area
        tzc_region region = {
            .filters          = TZC_CLUSTER_FILTER,
            .region           = 2,
            .region_base      = region_base,
            .region_top       = region_top,
            .sec_attr         = 0,
            .nsaid_permissions = TZC_REGION_ACCESS_RDWR(
                FVP_NSAID_CLUSTER0)
        };

        open(TZC_DEV_NAME, O_RDWR);
        ioctl(fd, TZC_CONFIGURE, &region);
    }

    TEEC_InvokeCommand(&sess, TA_TZCDEV_CMD, &op, &err_origin);

    INFO("shm.buffer[o]=%d\n", ((uint8_t *) shm.buffer)[0]);

    if (res != TEEC_SUCCESS)
        INFO("TEEC_InvokeCommand failed");
}

```

Listing 5.12: NA optionally preventing SW access to a region.

SW access allowed: We do not configure TZASC such that SW accesses are allowed by default. [Listing 5.13](#) shows the terminal output after the application terminates. We see that the TA is able to read and write to the shared memory region, and the NA observes the changes.

```
NW: shm.buffer[0]=111

TA: reading shm.buffer[0]
TA: shm.buffer[0]=111
TA: writing shm.buffer[0] -> 110
TA: shm.buffer[0]=110

NW: shm.buffer[0]=110
```

Listing 5.13: Output of allowing SW access to the shared memory region.

SW access prevented: We configure TZASC such that SW accesses to the shared memory region are prevented. [Listing 5.14](#) shows the terminal output after the application terminates. We see that the TA is unable to read from the shared memory region. This causes a synchronous external abort that is handled by OP-TEE core and results in the TA panicking (more details about this process are in [Section 4.2.1](#)). We see that the NA reads the same value as before. We additionally see that `TEEC_InvokeCommand` terminates with the error `TEEC_ERROR_TARGET_DEAD`, which indicates that the TA panicked.

```
NW: shm.buffer[0]=111

TA: reading shm.buffer[0]
OP-TEE Core: [abort] received synch external abort!
OP-TEE Core: [abort] abort in user mode (TA will panic)

NW: shm.buffer[0]=111
NW: TEEC_InvokeCommand failed with code TEEC_ERROR_TARGET_DEAD
```

Listing 5.14: Output of preventing SW access to the shared memory region.

RELATED WORK

We discuss related work in the field of TEE architectures and compare them to our design discussed in [Chapter 3](#). We mainly focus on TrustZone based hardware TEEs, however we also discuss other platforms and software-based virtualization architectures. We discuss ARM TEEs protecting CPU cores in [Section 6.1](#). We discuss ARM TEEs protecting both CPU cores and GPUs in [Section 6.2](#). We discuss Intel Software Guard Extensions (SGX) based TEEs in [Section 6.3](#). We discuss software-based virtualization architectures in [Section 6.4](#).

6.1 ARM CPU TEEs

We discuss ARM CPU TEEs Sanctuary [\[15\]](#), ReZone [\[17\]](#), and vTZ [\[22\]](#).

We based our TEE design heavily on the Sanctuary TEE. Sanctuary utilizes ARM TrustZone security features to allow for the creation of multiple Sanctuary instances within NW running Sanctuary Applications (SAs). This is reminiscent of NI-Ws present in NW running NI-As. Similar to NI-As, SAs are strictly isolated from NAs and the NOS by utilizing the TZASC. Sanctuary however does not provide security for GPUs and only provides confidentiality and integrity for applications running on CPU cores. Additionally, during Sanctuary instance initialization, a core is shut down and removed from the resources available to NOS, while the remaining cores stay under control of NOS. The Sanctuary instance then runs on the core after it has been restarted following additional initialization steps. After SA execution has completed, the core is again shut down before being restarted and reassigned to the NOS. The steps of shutting down the core in both

Sanctuary instance setup and teardown presents significant overheads, as highlighted in the Sanctuary paper. Our design instead utilizes process management by running a NI-W handler process on a separate core from NA that will call NOS.KM to setup the NI-W. We therefore do not undergo overheads due to shutting down and restarting a core.

ReZone takes a different approach of restructuring traditional ARM TrustZone based TEE architectures by partitioning the SW into multiple isolated domains with access only to private resources. This is in contrast with our design where we run NI-As within NW isolated compartments. ReZone does not trust the components present in SW, namely TAs and the TOS. This is different from our design where TAs and the TOS are part of the TCB. Since the TOS runs in s-EL1 and is by default given full permission to access both secure and NS memory regions, a compromised TOS can control the entire system. TZASC used in our design cannot be used to restrict memory access of an untrusted TOS since s-EL1 code has sufficient privileges to configure TZASC and revert the restriction. ReZone instead relies on a platform resource controller, which only allows specific bus masters to configure memory region permissions. A microcontroller embedded within a system SoC is trusted and used to configure the platform resource controller. ReZone, on the other hand, does not provide security for GPUs and only provides confidentiality and integrity for applications running on CPU cores. ReZone can complement our TEE design by reducing the privileges of SW software, thus further reducing the TCB for an NI-A.

vTZ virtualizes ARM TrustZone to be leveraged by multiple virtual machines. vTZ provides each guest virtual machine with a guest TEE, where both run in NW. vTZ relies on a NW hypervisor to virtualize functionality of guest TEEs by utilizing physical TrustZone protections. Memory protection between guest virtual machines and guest TEEs is done in SW by controlling page translation tables. TrustZone memory partitioning, such as by TZASC, is virtualized using a trap and emulate

technique. Our design is quite different from vTZ and does not use hypervisors. We instead focus on securing GPU-accelerated applications and we do not require large hypervisors to achieve that. Additionally, vTZ does not provide security for GPUs.

6.2 ARM GPU TEEs

We discuss a ARM GPU TEE SecDeep [32].

SecDeep aims at achieving secure GPU-accelerated ML inference with ARM TrustZone. SecDeep splits a ML model computation into a confidential computing base and a nonconfidential computing base. The confidential computing base runs in SW while the nonconfidential computing base runs in NW. Additionally, SecDeep provides secure GPU execution by moving the GPU device driver into the TOS. SecDeep therefore takes a hybrid approach of using both NW and SW during application execution. This is different from our TEE design where we run NI-As entirely within NW, which increases the total computation size possible and reduces the number of context switches that occur during application execution. Additionally, our design does not move the GPU device driver to SW, which is difficult due to typical driver sizes.

6.3 OTHER PLATFORMS

We discuss Intel SGX based TEEs Graviton [39] and HIX [25].

Intel SGX provides hardware-enforced code and data isolation [24]. SGX enclaves run security-sensitive code that can be verified with local and remote attestation. Once an enclave has been initialized, code and data within the enclave is isolated from the rest of the system, including the NOS. SGX however is tailored towards Intel x86 desktop and server

devices and not edge systems, where ARM TrustZone is instead used.

Graviton supports TEEs on discrete GPUs by utilizing Intel SGX. Graviton builds a TEE consisting of a secure context where GPU resources are cryptographically bound and isolated from untrusted NAs and the NOS. Graviton achieves this by preventing the GPU device driver from directly accessing GPU sensitive resources and instead routes all requests through the GPU command processor, which is extended to support security-related tasks.

HIX provides secure remote execution on discrete GPUs by utilizing Intel SGX, similar to Graviton. HIX modifies the PCIe interconnect and moves the GPU device driver to within a SGX enclave. Additionally, GPU-accelerated applications run within a SGX enclave and securely communicate with the GPU device driver enclave. This has some similarities with our design, where we invoke SW services during setup and teardown and also move the GPU device driver from NOS.

Both Graviton and HIX architectures are Intel-specific and do not apply to ARM architectures. Edge systems use integrated GPUs and share system memory with the CPU (discussed further in [Section 2.4.1](#)). The underlying hardware primitives, Intel SGX instead of ARM TrustZone, also limits the applicability of the designs towards edge systems. Additionally, edge systems are more resource constraint than desktop and server GPUs.

6.4 VIRTUALIZATION & SOFTWARE-BASED ISOLATION

We discuss software-based isolation techniques Hafnium [29] and pKVM [18, 19].

pKVM is currently being developed by Google to provide a common hypervisor for Android, by utilizing a small TCB between the Linux kernel and other services. pKVM is built as a component of Linux. During boot, the Linux kernel hands over control to the EL2 hypervisor which runs the pKVM code. pKVM handles memory mapping for itself and stage 2 page tables. Afterwards, the Linux kernel runs as the principal guest at EL1. Other services can run as other guests. Protection between the Linux kernel and guest services is enforced by the hypervisor. Context switching and inter-guest communication is done by hypervisor calls.

Hafnium provides memory isolation between a set of security domains by utilizing a hypervisor running in EL2. A set of virtual machines run in EL1. Hafnium makes a distinction between a primary virtual machine, typically running the main user-facing NOS, and a number of secondary virtual machines which provide services to the primary virtual machine. The isolation mechanisms done by the hypervisor are similar to pKVM.

Our design is quite different from pKVM and Hafnium and does not use hypervisors. We instead focus on securing GPU-accelerated applications by utilizing ARM TrustZone features. In our design, the performance of the NOS is not influenced when NI-W instances are executing in parallel since no hypervisor is running underneath the NOS. Additionally, neither pKVM nor Hafnium provide security for integrated GPUs.

CONCLUSION & FUTURE WORK

We provide a summary of our contributions in [Section 7.1](#). We discuss limitations of our work in [Section 7.2](#). We discuss future work in [Section 7.3](#).

7.1 CONTRIBUTIONS SUMMARY

We designed a TEE architecture for edge devices spanning both the CPU and GPU, while ensuring performant applications with minimal overheads due to the security features added. The TEE provides an isolation environment built in NW. The TEE additionally provides secure request-level GPU access. We utilized the TrustZone provided TZASC to securely partition memory regions. We implemented the necessary software changes and minimal hardware changes required to support a full implementation of the TEE. We performed hardware changes using ARM fast models due to limitations in currently available commercial hardware. We comprehensively evaluated specific overheads that will be encountered in the full TEE implementation. We constructed various scenarios to compare and evaluate overheads present. This provided a more general and detailed overview of the performance differences observed when utilizing TrustZone provided features. We additionally evaluated the security of the TEE by providing both a theoretical analysis and an empirical analysis. Overall, the evaluation ensured practicality of implementing the full TEE.

7.2 LIMITATIONS

As we discussed in [Section 2.5.3](#), current available hardware platforms have limitations in providing both a TrustZone with a TZASC and an integrated GPU with compute units. We were therefore forced to switch to a simulation environment to conduct testing. Although ARM fast models provide a functionally accurate simulation environment, They are not cycle accurate. Although we were still able to achieve relative timing comparisons using the simulated time, a more thorough timing analysis on either cycle accurate simulation models or on real hardware are needed. Additionally, due to the complexities involved with obtaining a base environment to work on top of, we were unable to implement the full TEE design.

Our design depends upon NSAIDs and TZASC configurable regions. Unfortunately, current ARM TZASC architectures have strict limits on the number of NSAIDs and configurable regions that are available. As we discussed in [Section 2.1.3](#), up to 16 NSAIDs are available and up to 8 TZASC regions can be configured on TZC-400. This limits our design's ability to support a large number of concurrent NI-W instances. However, this is a limitation that can be resolved through simple hardware updates. Additionally, our design depends on a GPU hardware extension to enable multiple GPU NSAIDs. Again, this is resolved through simple hardware updates, but is regardless a limitation in the applicability of our design.

We assumed in our design that cores could be assigned unique NSAIDs. This is normally not the case and is instead done at the granularity of clusters. As we discussed in [Section 3.3](#), this does not affect the design, but limits the number of concurrent NI-W instances possible. Hardware updates assigning unique NSAIDs to cores would resolve this limitation.

7.3 FUTURE WORK

Due to the limitations in currently available hardware platforms, we were unable to implement and evaluate our design on real hardware. However, we can implement our complete design within a ARM fast models simulation environment. We can then perform comprehensive system evaluations, building on top of our existing evaluations done. We would use the same evaluation methodology done previously to evaluate the complete design implementation. This would provide the missing parts within our current evaluation. Additionally, assuming new hardware platforms become available with the needed components, we can implement and evaluate our design on actual hardware to obtain real-world performance results.

BIBLIOGRAPHY

- [1] 96Boards. *HiKey 960 SoC Reference Manual*. URL: https://github.com/96boards/documentation/blob/master/consumer/hikey/hikey960/hardware-docs/HiKey960_SoC_Reference_Manual.pdf.
- [2] AMD. *AMD Radeon™ RX Graphics Cards*. URL: <https://www.amd.com/en/graphics/radeon-rx-graphics>.
- [3] ARM. *ARM Architecture Reference Manual for A-profile architecture*. URL: <https://developer.arm.com/documentation/ddi0487/latest>.
- [4] ARM. *ARM CoreLink TZC-400 TrustZone Address Space Controller Technical Reference Manual rop1*. URL: <https://developer.arm.com/documentation/ddi0504/c/>.
- [5] ARM. *ARM Mali Bifrost and Valhall OpenCL Developer Guide Version 4.8*. URL: <https://developer.arm.com/documentation/101574/latest>.
- [6] ARM. *ARM-software/nomali-model: A simple Mali 6xx/7xx register interface model that doesn't do any rendering*. URL: <https://github.com/ARM-software/nomali-model>.
- [7] ARM. *Fast Models Fixed Virtual Platforms (FVP) Reference Guide*. URL: <https://developer.arm.com/documentation/100966/latest/>.
- [8] ARM. *Fast Models Reference Guide*. URL: <https://developer.arm.com/documentation/100964/1117/>.
- [9] ARM. *Fast Models Reference Manual Version 11.16*. URL: <https://developer.arm.com/documentation/100964/1116/Fast-Models-components>.

- [10] ARM. *Fast Models User Guide*. URL: <https://developer.arm.com/documentation/100965/latest>.
- [11] ARM. *Learn the architecture: AArch64 Exception model*. URL: <https://developer.arm.com/documentation/102412/0102>.
- [12] ARM. *Mali GPU User-Space Binary Drivers*. URL: <https://developer.arm.com/tools-and-software/graphics-and-gaming/mali-drivers/user-space>.
- [13] ARM. *SMC Calling Convention (SMCCC)*. URL: <https://developer.arm.com/documentation/den0028/latest>.
- [14] ARM. *The Bifrost Shader Core*. URL: <https://developer.arm.com/documentation/102546/0100/Third-Gen-Mali-GPU-Architecture>.
- [15] Ferdinand Brasser, David Gens, Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stapf. "SANCTUARY: ARMing Trust-Zone with User-space Enclaves." In: Jan. 2019. DOI: [10.14722/ndss.2019.23448](https://doi.org/10.14722/ndss.2019.23448).
- [16] Keyan Cao, Yefan Liu, Gongjie Meng, and Qimeng Sun. "An Overview on Edge Computing Research." In: *IEEE Access* 8 (2020), pp. 85714–85728. DOI: [10.1109/ACCESS.2020.2991734](https://doi.org/10.1109/ACCESS.2020.2991734).
- [17] David Cerdeira, José Martins, Nuno Santos, and Sandro Pinto. "ReZone: Disarming TrustZone with TEE Privilege Reduction." In: *Proceedings of 31st USENIX Security Symposium (USENIX Security'22)*. 2022.
- [18] Will Deacon. *Virtualization for the Masses: Exposing KVM on Android*. URL: https://www.youtube.com/watch?v=wY-u6n75iXc&ab_channel=TheLinuxFoundation/.
- [19] Jake Edge. *KVM for Android*. URL: <https://lwn.net/Articles/836693/>.
- [20] GlobalPlatform. *GlobalPlatform Homepage*. URL: <https://globalplatform.org/>.

- [21] GlobalPlatform. *GlobalPlatform TEE Client API Specification*. URL: <https://globalplatform.org/specs-library/tee-client-api-specification/>.
- [22] Zhichao Hua, Jinyu Gu, Yubin Xia, Haibo Chen, Binyu Zang, and Haibing Guan. "vTZ: Virtualizing ARM TrustZone." In: *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, Aug. 2017, pp. 541–556. ISBN: 978-1-931971-40-9. URL: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hua>.
- [23] OpenCL Overview The Khronos Group Inc. *The Bifrost Shader Core*. URL: <https://www.khronos.org/opencl/>.
- [24] Intel. *Intel® Software Guard Extensions*. URL: <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html>.
- [25] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. "Heterogeneous Isolated Execution for Commodity GPUs." In: *ASPLOS '19*. New York, NY, USA: Association for Computing Machinery, 2019, pp. 455–468. ISBN: 9781450362405. DOI: [10.1145/3297858.3304021](https://doi.org/10.1145/3297858.3304021).
- [26] Jongmin Jo, Sucheol Jeong, and Pilsung Kang. "Benchmarking GPU-Accelerated Edge Devices." In: *2020 IEEE International Conference on Big Data and Smart Computing (BigComp)*. 2020, pp. 117–120. DOI: [10.1109/BigComp48618.2020.00-89](https://doi.org/10.1109/BigComp48618.2020.00-89).
- [27] Linaro. *FVP – OP-TEE*. URL: <https://optee.readthedocs.io/en/latest/building/devices/fvp.html>.
- [28] Linaro. *FVP — OP-TEE documentation*. URL: <https://optee.readthedocs.io/en/latest/building/devices/fvp.html>.
- [29] Linaro. *Hafnium - Trusted Firmware*. URL: <https://www.trustedfirmware.org/projects/hafnium/>.
- [30] Linaro. *OP-TEE Documentation*. URL: <https://optee.readthedocs.io/en/latest/>.

- [31] Linaro. *Trusted Firmware-A Documentation*. URL: <https://trustedfirmware-a.readthedocs.io/en/latest/>.
- [32] Renju Liu, Luis Garcia, Zaoxing Liu, Botong Ou, and Mani Srivastava. "SecDeep: Secure and Performant On-Device Deep Learning Inference Framework for Mobile and IoT Devices." In: *Proceedings of the International Conference on Internet-of-Things Design and Implementation*. IoTDI '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 67–79. ISBN: 9781450383547. DOI: [10.1145/3450268.3453524](https://doi.org/10.1145/3450268.3453524).
- [33] NVIDIA. *Jetson Download Center*. URL: <https://developer.nvidia.com/embedded/downloads>.
- [34] NVIDIA. *NVIDIA TITAN RTX*. URL: <https://www.nvidia.com/en-us/deep-learning-ai/products/titan-rtx/>.
- [35] Bernard Ngabonziza, Daniel Martin, Anna Bailey, Haehyun Cho, and Sarah Martin. "TrustZone Explained: Architectural Features and Use Cases." In: *2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC)*. 2016, pp. 445–451. DOI: [10.1109/CIC.2016.065](https://doi.org/10.1109/CIC.2016.065).
- [36] Zhenyu Ning, Jinghui Liao, Fengwei Zhang, and Weisong Shi. "Preliminary Study of Trusted Execution Environments on Heterogeneous Edge Platforms." In: *2018 IEEE/ACM Symposium on Edge Computing (SEC)*. 2018, pp. 421–426. DOI: [10.1109/SEC.2018.00057](https://doi.org/10.1109/SEC.2018.00057).
- [37] Sandro Pinto and Nuno Santos. "Demystifying Arm TrustZone: A Comprehensive Survey." In: *ACM Comput. Surv.* 51.6 (2019). ISSN: 0360-0300. DOI: [10.1145/3291047](https://doi.org/10.1145/3291047). URL: <https://doi.org/10.1145/3291047>.
- [38] David Reinsel-John Gantz-John Rydning. "The digitization of the world from edge to core." In: *Framingham: International Data Corporation* (2018), p. 16.

- [39] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. “Graviton: Trusted Execution Environments on GPUs.” In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 681–696. ISBN: 978-1-939133-08-3. URL: <https://www.usenix.org/conference/osdi18/presentation/volos>.